

SE I - Projektdokumentation

EnergyBalance (F2)

, Vince Hohfeld, Armando Stefanow, Omar Al Jawabreh, Tomàs Kuna, Hoang Son
David Tran, Lewin Hantzsch, Anwar Ahmad Alissa Aljasim

05. Juli 2026

Inhalt

Teamspezifikation	1
1. Team Charter: F2 - EnergyBalance	2
1.1. Über uns	2
1.2. Zweck	2
1.3. Rollen	2
1.4. Ansprechpartner	2
1.5. Meetings	2
1.6. Kommunikation	3
1.7. Arbeitsweise	3
1.8. Teamvereinbarungen	4
Produktspezifikation	7
2. Produktkonzept: F2 - EnergyBalance	8
2.1. Motivation	8
2.2. Produktvision	8
2.3. Produktpositionierung	9
2.4. Stakeholder	10
2.5. Hauptfunktionen	10
2.6. Qualitätsziele & Einschränkungen	12
2.7. Versionsverlauf	14
3. Domänenmodell und Glossar: F2 - EnergyBalance	15
3.1. Glossar	15
3.2. Domänenmodell	26
4. UX-Konzept: F2 - EnergyBalance	31
4.1. Personas	31
4.2. Nutzungskontext	32
4.3. Systemweite Interaktionsflüsse	33
4.4. Designsystem & Interaktionsprinzipien	36
Technische Spezifikation	38
5. Architekturdokument: F2 - EnergyBalance	39
5.1. Architektursichten	39
5.2. Technologie-Stack	43
5.3. Architekturrelevante Anforderungen	45
5.4. Architekturentscheidungen	47
5.5. Risiken & technische Schulden	52
6. Softwaredesign: F2 - EnergyBalance	55
6.1. Systemübersicht & Schichtenarchitektur	55
6.2. Komponentenübersicht	56
6.3. Wichtige Interaktionsflüsse	58

6.4. Datenmodell	60
6.5. Eingesetzte Muster & Konventionen	62
7. Testkonzept: F2 - EnergyBalance	65
7.1. Teststrategie	65
7.2. Testarten	65
7.3. Nicht getestet	66
7.4. Werkzeuge & Frameworks	66
7.5. Testabdeckung	67
7.6. Integration in den Entwicklungsprozess	67
7.7. Fehlerbehandlung & Bugtracking	68
8. Deployment & Betrieb: F2 - EnergyBalance	69
8.1. Umgebungsübersicht	69
8.2. Lokale Entwicklungsumgebung	69
8.3. Docker & Container	71
8.4. CI/CD-Pipeline	72
8.5. Deployment-Ablauf	73
8.6. Rollback	74
8.7. Monitoring & Logging	75

Teamspezifikation

- Teamcharta

1. Team Charter: F2 - EnergyBalance

1.1. Über uns

Wir sind ein studentisches Entwicklungsteam aus den Studiengängen: Wirtschaftsinformatik, Allgemeine Informatik und Verwaltungsinformatik. Der Auftrag ist die Entwicklung einer nutzerzentrierten Softwarelösung. Dabei liegt unser Fokus darauf, reale Probleme aus dem Studienalltag zu identifizieren und mit einer praxisnahen Anwendung zu lösen.

1.2. Zweck

Was bauen wir, und warum?

Mit EnergyBalance entwickeln wir eine Anwendung, die Studierende dabei unterstützt, ihre mentale und zeitliche Belastung im Studienalltag bewusst zu erfassen, zu analysieren und aktiv zu steuern. Ziel ist es, Zusammenhänge zwischen Aufgaben, Stress und Energielevel sichtbar zu machen, um Überbelastung frühzeitig zu erkennen und eine bessere Balance zwischen Produktivität und Wohlbefinden zu erreichen.

1.3. Rollen

Rolle	Person
Product Owner	Vince Hohfeld
Scrum Master	Armando Stefanow
Tech Lead ^[1]	Hoang Son David Tran
Developers	Anwar Ahmad Alissa Alijasim, Lewin Hantzsch, Omar Al Jawabreh, Hoang Son David Tran, Tomàs Kuna

1.4. Ansprechpartner

Funktion	Person
Coach	Marlene Winter
Modulverantwortlicher	Prof. Dr.-Ing. habil. Jürgen Anke

1.5. Meetings

Ereignis	Wann	Dauer	Moderation
Stand-up	Bei Bedarf, in der Regel montags in der Hochschule sowie zusätzlich über Discord	ca. 15-30 min	Scrum Master
Sprint Planning	Zu Beginn jedes Sprints	30–60 min	Scrum Master
Sprint Review	Am Ende jedes Sprints	30–60 min	Product Owner
Sprint Retrospektive	Im Anschluss an das Sprint Review	ca. 30 min	Scrum Master
Backlog Refinement	Regelmäßig während des Sprints, meist montags oder bei Bedarf über Discord	ca. 30 min	Product Owner

1.6. Kommunikation

Thema	Vereinbarung
Primärer Kanal	Discord (Server: SE_Projekt-Team_F2)
Entscheidungsprotokoll	GitHub Repository (AsciiDoc-Dokumentation und Issues); Miro Board für Brainstorming und Ideensammlung
Erwartete Reaktionszeit	Innerhalb von 24 Stunden, in dringenden Fällen möglichst schneller
Kernarbeitszeit	Keine festen Kernarbeitszeiten; Zusammenarbeit überwiegend montags in der Hochschule sowie nach Absprache über Discord
Videokonferenz-Tool	Discord (Voice-Channel)
Asynchrone Abstimmung	Discord (Textkanäle); WhatsApp für Terminabsprachen und kurze organisatorische Mitteilungen

1.7. Arbeitsweise

1.7.1. Definition of Ready (DoR)

Eine User Story ist bereit für die Entwicklung, wenn:

- ☐ Die User Story ist verständlich formuliert und beschreibt einen fachlichen Mehrwert.
- ☐ Die Akzeptanzkriterien sind klar und testbar beschrieben.
- ☐ Abhängigkeiten zu anderen User Stories oder technischen Stories sind bekannt.
- ☐ Offene fachliche Fragen wurden geklärt.
- ☐ Die Story ist ausreichend klein oder wurde bei Bedarf in kleinere User Stories aufgeteilt.
- ☐ Das Team schätzt die Story als umsetzbar innerhalb eines Sprints ein.

1.7.2. Definition of Done (DoD)

Eine User Story gilt als fertig, wenn:

- ☐ Die Implementierung ist abgeschlossen und in den Hauptbranch gemergt.
- ☐ Der Code wurde von mindestens einer weiteren Person überprüft (Code Review).
- ☐ Alle Akzeptanzkriterien sind erfüllt.
- ☐ Die Funktion wurde erfolgreich getestet.
- ☐ Die Dokumentation wurde bei Bedarf aktualisiert.
- ☐ Der Product Owner hat die User Story abgenommen.
- ☐ Das Feature ist vollständig integriert und lauffähig.
- ☐ Es bestehen keine bekannten Fehler mit hoher Priorität, die durch die Umsetzung entstanden sind.

1.7.3. Code & Versionierung

Thema	Vereinbarung
Branching-Strategie	Feature Branches mit Merge in den main -Branch über Pull Requests
PR-Review-Regeln	Mindestens ein Approval vor dem Merge
Merge-Berechtigung	Jedes Teammitglied nach Approval
Commit-Konventionen	Conventional Commits

1.8. Teamvereinbarungen

Diese Vereinbarungen haben wir gemeinsam getroffen:

1. Respektvoller Umgang und offene Kommunikation.
2. Probleme werden persönlich angesprochen und nicht über Chat oder Dritte diskutiert.
3. Fehler sind Lernchancen und keine Schuldzuweisungen.
4. Meetings beginnen pünktlich. Verhinderungen werden rechtzeitig kommuniziert. Termine die das Treffen verhindern, werden im Voraus angekündigt (z.B. Arzttermine).
5. Entscheidungen werden dort dokumentiert, wo alle sie finden können. Jeder hat das Recht Ent-

scheidungen zu hinterfragen und Verbesserungsvorschläge einzubringen.

6. Keiner wird unterdrückt oder ausgeschlossen. Alle Teammitglieder haben die gleichen Rechte und Pflichten.

Diese Teamcharta wird bei Bedarf sowie nach Abschluss eines Sprints gemeinsam überprüft und aktualisiert.

[1] optionale Rolle, aber empfohlen, um technische Entscheidungen zu koordinieren

Produktspezifikation

- Produktkonzept
- Domänenmodell und Glossar
- UX-Konzept

2. Produktkonzept: F2 - EnergyBalance

Dieses Dokument beschreibt das Produkt auf konzeptioneller Ebene — was gebaut wird, warum, für wen, und unter welchen Rahmenbedingungen. Es richtet sich an alle Projektbeteiligten: Stakeholder, Product Owner, Designer und Entwickler:innen.

Das Dokument ist so aufgebaut, dass es relativ stabil bleibt. Detaillierte UX-Artefakte wie Personas, Nutzerkontext, Interaktionsflüsse und Design-Entscheidungen sind im UX-Konzept beschrieben.

2.1. Motivation

Warum wird dieses Produkt gebaut? Welches Problem existiert heute, und warum ist es relevant? Dieser Abschnitt beschreibt den Ausgangszustand und den Handlungsbedarf — ohne bereits eine Lösung zu nennen. 3–5 Sätze genügen.

Studierende stehen im Alltag häufig unter hoher mentaler und zeitlicher Belastung, ohne einen klaren Überblick über ihr eigenes Energielevel und Stressverhalten zu haben. Aufgaben, Prüfungen und Verpflichtungen werden meist isoliert betrachtet, wodurch Zusammenhänge zwischen Belastung und Leistungsfähigkeit schwer erkennbar sind. Das wiederum führt oft zu Überforderung, ineffizientem Zeitmanagement und fehlenden Erholungsphasen.

2.2. Produktvision

Die Produktvision beschreibt in einem oder wenigen Sätzen den angestrebten Zielzustand. Sie ist inspirierend, prägnant und stabil — sie sollte sich über die gesamte Projektlaufzeit nicht wesentlich verändern.

2.2.1. Visions-Statement

Für [Studierende], die [ihre mentale und zeitliche Belastung im Studienalltag besser verstehen und steuern möchten], ist [EnergyBalance] ein [Selbstmanagement- und Tracking-Tool], das [Transparenz über Energielevel, Stress und Aufgaben schafft]. Im Gegensatz zu [To-Do oder Kalender-Apps] bietet unser Produkt [die Möglichkeit, Belastung und Wohlbefinden aktiv zu analysieren und daraus individuelle Verbesserungen abzuleiten].



Das Visions-Statement kann im Team mit der „Elevator Pitch“-Methode oder dem „Future Press Release“-Format erarbeitet werden.

2.2.2. Zielbild

Optional: Beschreibe in 2–4 Sätzen, wie die Welt mit diesem Produkt aussieht. Was hat sich für die Nutzer:innen verändert?

Studierende nutzen EnergyBalance als festen Bestandteil ihres Alltags, um ihre Belastung bewusst wahrzunehmen und langfristig besser zu steuern. Sie erkennen Muster in ihrem Verhalten, vermeiden Überbelastung und planen ihre Aufgaben realistischer unter Berücksichtigung ihres Energielevels. Das Ergebnis ist ein ausgewogenes Verhältnis zwischen Produktivität und Wohlbefinden, sowie eine nachhaltige Verbesserung der Selbstorganisation im Studium.

2.3. Produktpositionierung

Wer sind die Zielgruppen, und wie positioniert sich das Produkt im Vergleich zu bestehenden Alternativen?

2.3.1. Zielgruppen

Zielgruppe	Beschreibung
Studierende	Studierende, die ihre mentale und zeitliche Belastung im Studienalltag besser verstehen und steuern möchten. Ihre Motivation besteht darin, Überforderung zu vermeiden, produktiver zu arbeiten und gleichzeitig ihr Wohlbefinden zu verbessern.
Studienanfänger	Studienanfänger, die Schwierigkeiten haben, sich an die neue Struktur und Anforderungen des Studiums anzupassen. Sie benötigen Unterstützung bei der Selbstorganisation und beim Aufbau gesunder Arbeitsgewohnheiten.
Studienberater/Coaches	Beratende Personen, die Studierende bei der Bewältigung ihres Studienalltags unterstützen. Sie nutzen aggregierte Daten, um gezielt Hilfestellungen zu geben und Überlastung frühzeitig zu erkennen.



Ausführliche Beschreibungen der Nutzenden — Personas, Nutzungskontext, typische Aufgaben — sind im [UX-Konzept](#) dokumentiert.

2.3.2. Wettbewerb & Alternativen

Alternative	Beschreibung	Warum unser Produkt besser passt
Klassische To-Do-Apps (z.B. Kalender, Notizen-App)	Nutzer organisieren ihre Aufgaben und Termine über Kalender oder To-Do-Listen. Dabei werden jedoch ausschließlich Aufgaben erfasst, ohne die persönliche Belastung oder das Energielevel zu berücksichtigen.	EnergyBalance geht über die reine Aufgabenorganisation hinaus, indem es zusätzlich das individuelle Energielevel und Stressverhalten erfasst. Dadurch können Zusammenhänge zwischen Aufgaben und Belastung erkannt und aktiv gesteuert werden.

Alternative	Beschreibung	Warum unser Produkt besser passt
Eigenes Bauchgefühl/ keine systematische Erfassung	Viele Studierende verlassen sich auf ihr subjektives Empfinden, um ihre Belastung einzuschätzen. Eine strukturierte Dokumentation oder Analyse findet nicht statt.	EnergyBalance ermöglicht eine objektive und langfristige Analyse durch kontinuierliches Tracking. Muster und Überlastungssituationen werden sichtbar, die sonst eher unbemerkt bleiben würden.
Fitness- und Gesundheits-Apps (z.B. Fitbit, Apple Health)	Einige Apps erfassen allgemeine Gesundheitsdaten wie Schlaf oder Bewegung, sind jedoch nicht speziell auf den Stress im Studienalltag und akademische Anforderungen ausgerichtet.	EnergyBalance ist gezielt auf den Kontext Studium zugeschnitten und verknüpft Belastung direkt mit Aufgaben, Lernphasen und Verpflichtungen. Dadurch entsteht ein konkreter Mehrwert für Studierende.

2.4. Stakeholder

Wer hat Einfluss auf das Produkt oder ein Interesse an dessen Erfolg? Stakeholder formen oft Anforderungen und Rahmenbedingungen — ihre Nennung hier erklärt, warum bestimmte Entscheidungen getroffen wurden.

Stakeholder	Rolle / Interesse	Erwartungen an das Produkt	Einfluss
Studierende	möchten ihre Belastung, Energielevel und ihren Studienalltag besser verstehen und optimieren	Einfache Bedienung, schnelle Eingabe von Daten, aussagekräftige Auswertungen und konkrete Verbesserungsvorschläge	Hoch
Studienberater/Coaches	Unterstützen Studierende bei Problemen im Studium und möchten frühzeitig Überlastung erkennen	Zugriff auf aggregierte Daten (nur mit Zustimmung), verständliche Auswertungen und Hinweise auf kritische Entwicklungen	Niedrig
Hochschule/Verwaltung	Interesse an der Förderung des Studienerfolgs und der Reduktion von Studienabbrüchen	Einhaltung der DSGVO sowie einer datenschutzkonformen Systemarchitektur.	Mittel
Lehrende	Profitieren von besser organisierten und weniger überlasteten Studierenden	Indirekte Verbesserung der Studienleistung und Teilnahme	Niedrig

2.5. Hauptfunktionen

Welche Kernfunktionen muss das Produkt bieten, um die Produktvision zu erfüllen? Jede Funktion

ist direkt auf einen Nutzerbedarf zurückgeführt — nicht als technische Anforderung, sondern als Antwort auf ein Nutzungsproblem.

Diese Hauptfunktionen sollten als Epics im Product Backlog für die Gruppierung von User Stories verwendet werden.

Nutzerbedarf	Hauptfunktion	Priorität
Studierende wollen ihr aktuelles Stress- und Energielevel einfach und regelmäßig erfassen können.	F1 - Eingabe von Stress- und Energiedaten	Must Have
Studierende wollen ihre Belastungs- und Energiewerte übersichtlich visualisieren und Entwicklungen erkennen können.	F2 - Datenvisualisierung	Must Have
Studierende wollen wichtige Hinweise und Warnungen zu ihrer Belastung erhalten.	F3 - Benachrichtigungsanzeige	Should Have
Studierende wollen beim Einstieg unterstützt werden und individuelle Referenzwerte für ihre Auswertungen erhalten.	F4 - Onboarding und Baseline-Ermittlung	Should Have
Studierende wollen frühzeitig auf kritische Belastungssituationen aufmerksam gemacht werden.	F5 - Frühwarnsystem	Should Have
Studierende wollen personalisierte Empfehlungen zur Verbesserung ihres Studien- und Arbeitsalltags erhalten.	F6 - Empfehlungssystem	Should Have
Studierende wollen ihre persönliche Entwicklung reflektieren und subjektive Eindrücke dokumentieren können.	F7 - Reflexion und Selbstbewertung	Should Have
Studierende wollen die volle Kontrolle über ihre Daten behalten und ihre Privatsphäre schützen.	F8 - Datenschutz und Kontrolle	Must Have
Das System soll alle fachlichen Daten konsistent und einheitlich verwalten können.	F9 - Zentrales Datenmodell	Technische Basis
Studierende wollen Aufgaben und Termine mit ihrer Belastung in Beziehung setzen können.	F10 - Aufgaben- und Kontextmodell	Should Have
Studierende wollen ihre Daten statistisch auswerten und Muster erkennen können.	F11 - Datenanalyse	Should Have

Nutzerbedarf	Hauptfunktion	Priorität
Studierende wollen zusätzliche Einflussfaktoren wie Schlaf, Arbeit oder körperliche Aktivität erfassen können.	F12 - Erweiterte Kontextdaten	Could Have
Studierende wollen ihre Daten dauerhaft speichern und jederzeit abrufen können.	F13 - Datenspeicherung	Must Have
Studierende wollen die Anwendung personalisiert mit Konto oder alternativ lokal ohne Konto nutzen können.	F14 - Benutzerverwaltung	Must Have
Studierende wollen ihre Daten exportieren und gezielt mit anderen Personen teilen können.	F15 - Export und Datenfreigabe	Could Have
Studierende wollen zwischen den verschiedenen Ansichten der PWA navigieren können.	F16 - Routing und Navigation	Must Have



Die angegebenen Prioritäten spiegeln den derzeitigen Stand der MVP-Planung wider und werden im Rahmen der weiteren Produkt- und Backlog-Entwicklung kontinuierlich überprüft und angepasst.

2.6. Qualitätsziele & Einschränkungen

Dieser Abschnitt beschreibt, unter welchen Rahmenbedingungen das Produkt entwickelt und betrieben wird. Es wird unterschieden zwischen Qualitätszielen (angestrebte Eigenschaften) und Einschränkungen (nicht verhandelbare Rahmenbedingungen).

2.6.1. Qualitätsziele

Qualitätsziele sind messbare oder zumindest überprüfbare Eigenschaften, die das Produkt erfüllen soll. Sie sollten spezifisch genug sein, um als Testkriterium zu dienen.



Dieses Dokument ist die einzige Quelle für Qualitätsziele und Einschränkungen. Das [Architektur](#)-Dokument referenziert diese Anforderungen und dokumentiert die technischen Entscheidungen, die sich daraus ergeben — ohne sie hier zu wiederholen.

Qualitätsziel	Beschreibung	Messkriterium
Benutzbarkeit	Das Produkt soll intuitiv und ohne lange Einarbeitung nutzbar sein.	Nutzer können ihr Energie- und Stresslevel mit maximal 3 Klicks und in unter 15 Sekunden erfassen.

Qualitätsziel	Beschreibung	Messkriterium
Performance	Das System soll schnell auf Eingaben und Abfragen reagieren.	Ladezeiten unter 2 Sekunden bei normalen Nutzerinteraktionen.
Verfügbarkeit	Das System soll während der üblichen Nutzungszeiten zuverlässig erreichbar sein.	Die Anwendung ist innerhalb der Entwicklungs- und Testphase zuverlässig nutzbar. Außerdem gibt es keine ungeplanten Ausfälle während der Demonstration.
Datensicherheit	Sensible Nutzerdaten müssen geschützt gespeichert werden.	Zugriff erfolgt ausschließlich nach erfolgreicher Authentifizierung; Daten werden verschlüsselt übertragen.
Datenschutz	Nutzer müssen vollständige Kontrolle über ihre persönlichen Daten haben. Die App erfasst nur das Minimum an notwendigen Daten.	Nutzer können ihre eigenen Daten einsehen. Die Verarbeitung personenbezogener Daten erfolgt gemäß DSGVO.
Verständlichkeit	Die dargestellten Auswertungen müssen leicht interpretierbar sein.	Diagramme verfügen über eindeutige Achsenbeschriftungen, Legenden und sind ohne zusätzliche Erklärung verständlich.
Zuverlässigkeit der Daten	Erfasste Check-Ins dürfen nach erfolgreicher Speicherung nicht verloren gehen.	Gespeicherte Check-Ins können nach dem erneuten Laden der Anwendung vollständig abgerufen werden.
Akzeptanz & Motivation	Die Anwendung soll Studierende zu einer regelmäßigen Nutzung motivieren.	Die Erfassung eines Check-Ins ist mit wenigen Eingaben möglich und kann innerhalb kurzer Zeit abgeschlossen werden.

2.6.2. Einschränkungen

Einschränkungen sind Rahmenbedingungen, die nicht verhandelbar sind — technische, organisatorische, rechtliche oder zeitliche Vorgaben.

Einschränkung	Beschreibung	Kategorie
Projektlaufzeit	Das Produkt wird in zwei Semestern umgesetzt, gegliedert in Sprints. Jeder Sprint endet mit einem Review.	Zeitlich
DSGVO-Konformität	Personenbezogene Daten dürfen nur gemäß DSGVO verarbeitet und gespeichert werden.	Rechtlich

Einschränkung	Beschreibung	Kategorie
Hochschulkontext	Das System ist auf Studierende und deren Alltag ausgerichtet.	Organisatorisch
GitHub & Scrum-Prozess	Versionskontrolle erfolgt ausschließlich über GitHub. Aufgaben werden als Issues im GitHub Project Board verwaltet. Der Scrum-Prozess (Sprint Planning, Review, Retrospective) ist verpflichtend.	Organisatorisch
Verantwortungsvoller KI-Einsatz	KI-Werkzeuge dürfen unterstützend eingesetzt werden, müssen aber transparent gemacht und kritisch geprüft werden. KI-Nutzung wird im Repository dokumentiert (gemäß Hochschulvorgabe).	Organisatorisch
Technische Infrastruktur	Frontend als React Progressive Web App, Backend mit Node.js/Nest sowie PostgreSQL über Supabase. Es wird keine Spezialhardware benötigt.	Technisch
Programmiersprache	Frontend: TypeScript mit React. Backend: JavaScript mit Node.js und Nest.	Technisch
Offline-Grundfähigkeit	Einträge müssen auch ohne aktive Internetverbindung erfassbar sein. Sobald eine Internetverbindung hergestellt wurde, sollen die Daten automatisch geupdated werden.	Technisch

2.7. Versionsverlauf

Version	Datum	Änderung
1.0	15.04.2026	Initiale Version nach Kick-off-Workshop
1.1	02.07.2026	Aktualisierung der Produktvision, Hauptfunktionen, Qualitätsziele und technischen Rahmenbedingungen entsprechend dem aktuellen Projektstand

Dieses Dokument ist bewusst stabil gehalten. Änderungen sollten nur bei wesentlichen Richtungsentscheidungen vorgenommen werden und im Versionsverlauf dokumentiert sein.

3. Domänenmodell und Glossar: F2 - EnergyBalance

Dieses Dokument beschreibt die wesentlichen fachlichen Begriffe und ihre Beziehungen untereinander. Es bildet die gemeinsame Sprache des Teams und aller Stakeholder — unabhängig von technischen Umsetzungsentscheidungen. Alle Begriffe in User Stories, Personas und Gesprächen mit Stakeholdern sollten hier definiert sein.

3.1. Glossar

Alle **fachlichen** Begriffe und Abkürzungen werden hier alphabetisch definiert. Ziel ist eine einheitliche Sprache im gesamten Team. Technische Implementierungsdetails gehören nicht hierher!

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Abweichung	Unterschied zwischen einem aktuellen Wert und einem Referenzwert, z.B. der persönlichen Baseline.	Synonym: Differenz, Abweichungswert. Abgrenzung: Eine Abweichung kann erhöht oder verringert sein. [US-021]
Aktivitätsdaten	Gespeicherte Daten zu körperlichen Aktivität, mindestens Aktivitätstyp, Dauer und Datum.	Synonyme: Bewegungsdaten, Aktivitätsinformationen. Abgrenzung: Aktivitätsdaten sind Kontextdaten, keine Stress- oder Energiewerte. [US-048].
Analyse	Auswertung vorhandener Daten, um Muster, Trends, Warnungen, Ursachen oder Empfehlungen abzuleiten.	Synonyme: Auswertung, Datenanalyse. Abgrenzung: Analyse ist nicht dasselbe wie Datenerfassung. [US-024]
Analyseansicht	Bereich der Anwendung, in dem gespeicherte Daten ausgewertet und visualisiert werden.	Abgrenzung: Nicht Eingabe-seite; dient der Auswertung, nicht der Erfassung. [US-005 und US-006]
Analysefunktion	Funktion, die Daten auswertet, z.B. für Trends, Warnungen oder Empfehlungen.	Synonym: Auswertungsfunktion. Abgrenzung: Datenerfassung bleibt auch ohne Analysefunktion möglich. [US-036]
Analyseoption	Einstellung, mit der festgelegt wird, welche Daten in Analysen berücksichtigt werden.	Synonyme: Analyseeinstellung, Auswertungsoption. [US-035]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Analysierte Daten	Daten, die für Auswertungen, Empfehlungen oder Analysen verwendet werden.	Synonyme: ausgewertete Daten. Abgrenzung: Nicht alle gespeicherten Daten müssen analysiert werden. [US-035]
Angemeldete Nutzer	Nutzer, der sich erfolgreich angemeldet hat und Zugriff auf seine eigenen Daten erhält.	Synonym: eingeloggter Nutzer. Abgrenzung: Nicht Gastnutzer. [US-033].
Anmeldung	Vorang, bei dem ein Nutzer mit gültigen Zugangsdaten Zugriff auf sein Benutzerkonto erhält.	Synonyme: Login, sichere Anmeldung. [US-054]
Aufgabe	Verpflichtung oder Tätigkeit eines Studierenden, die erfasst, geplant und mit Belastung oder Stresswerten verknüpft werden kann.	Synonyme: To-Do, Tätigkeit. [US-037]
Aufgabendaten	Gespeicherte Informationen zu Aufgaben, die für Analyse, Zeitplanung oder Ursachenbestimmung genutzt werden.	Synonyme: Aufgabeninformationen. Abgrenzung: Aufgabe = fachliches Objekt. Aufgabendaten = gespeicherte Information dazu. [US-025].
Aufgabenkategorie	Thematische Einordnung einer Aufgabe	Synonyme: Kategorie. Abgrenzung: Kategorie beschreibt Art/Thema; Priorität beschreibt Wichtigkeit. [US-038]
Aufgabenplanung	Planung von Aufgaben anhand von Zeitaufwand, Deadline, Priorität und Belastung	Synonyme: Zeitplanung, Planung. Abgrenzung: Aufgabenplanung ist konkreter als allgemeine Zeitplanung. [US-040]
Ausgeschlossene Daten	Daten, die aufgrund einer Nutzereinstellung nicht in Analysen berücksichtigt werden.	Synonyme: nicht berücksichtigte Daten. Abgrenzung: Die Daten werden nicht gelöscht, sondern nur nicht analysiert. [US-035]
Auslöser der Warnung	Fachlicher Grund, warum eine Warnung erzeugt wurde.	Synonyme: Warnungsgrund, Ursache der Warnung. [US-020 und US-022]
Baseline	Persönlicher Referenzwert oder Referenzbereich für typische Stress- und Energiewerte.	Synonyme: Normalwert, Referenzwert, normales Niveau. [US-016]
Baseline-Ermittlung	Berechnung oder Festlegung einer persönlichen Baseline aus vorhandenen Daten.	Synonyme: Baseline-Berechnung. [US-018]
Baseline-Vergleich	Vergleich aktueller Stress- oder Energiewerte mit der persönlichen Baseline.	Synonym: Vergleich mit persönlicher Baseline. [US-042]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Belastung	Mentale oder zeitliche Beanspruchung eines Studierenden durch Aufgaben, Termine, Stress, Energie oder Kontextfaktoren	Abgrenzung: Belastung ist komplexer als Stresslevel. Stresslevel ist ein Messwert zur Belastung. [US-009].
Belastungsdaten	Daten, die zur Bewertung der Belastung eines Studierenden verwendet werden.	Synonyme: Belastungswerte, Belastungsinformationen.
Belastungsspitze	Plötzlicher oder außergewöhnlich hoher Belastungswert, der kritisch sein kann.	Synonyme: Belastungsspeak, kritischer Belastungswert. [US-020].
Belastungstreiber	Faktor, Aufgabe oder Kontext, der die Belastung eines Studierenden erhöht oder wesentlich beeinflusst.	Synonyme: Stressursache, Einflussfaktor. [US-025]
Belastungstrend	Entwicklung der Belastung über einen Zeitraum.	Synonyme: Belastungsentwicklung, Abgrenzung: Spezieller als allgemeiner Trend. [US-024]
Benachrichtigung	Meldung innerhalb der App, die über Hinweise, Warnungen oder wichtige Veränderungen informiert.	Synonyme: Meldung, Nachricht, Hinweise und Warnungen sind Arten von Benachrichtigungen. [US-011]
Benachrichtigungseinstellung	Nutzereinstellung, mit der Benachrichtigungen aktiviert, deaktiviert oder gefiltert werden.	Synonyme: Benachrichtigungskonfiguration. [US-015]
Benachrichtigungshistorie	Sammlung vergangener Benachrichtigungen, die chronologisch angezeigt werden.	Synonyme: Historie, vergangene Benachrichtigungen [US-014]
Benachrichtigungstyp	Kategorie einer Benachrichtigung, z.B. Warnung oder Hinweis	Synonym: Benachrichtigungskategorie. [US-013]
Benutzerkonto	Persönlicher Account eines Studierenden, dem gespeicherte Daten zugeordnet sind.	Synonyme: Nutzerkonto, Account. [US-053]
Benutzerprofil	Bereich oder Datensatz eines Benutzerkontos, in dem Profilinformationen und Einstellungen verwaltet werden.	Synonyme: Profil, Nutzerprofil. [US-055]
Bewertungsskala	Skala, mit der eine Tagesbewertung oder Reflexion strukturiert erfasst wird.	Abgrenzung: Nicht Stressskala oder Energieskala. [US-028]
Datenabruf	Laden gespeicherter Daten, damit diese erneut eingesehen oder verwendet werden können.	Synonyme: Daten laden, Abruf gespeicherter Daten. [US-050]
Datenexport	Export gespeicherter Nutzerdaten in einem wählbaren Datenformat.	Synonyme: Export. Abgrenzung: Export kopiert Daten; er löscht oder verändert die Originaldaten nicht. [US-057]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Datenfreigabe	Kontrollierte Weitergabe ausgewählter Daten an andere Personen oder Anwendungen	Synonyme: Daten teilen, Freigabe. [US-059]
Datenkategorie	Gruppe von Daten, die getrennt betrachtet, ausgewählt oder exportiert werden kann	Synonym: Datentyp. Für den Export eher Datenkategorie, für Datenschutz eher Datentyp. [US-058].
Datenpunkt	Einzelner Wert oder Messpunkt innerhalb eines Diagramms.	Synonyme: Messpunkt, Diagrammpunkt. [US-007]
Datenspeicherung	Funktion, durch die erfasste Daten dauerhaft erhalten bleiben.	Synonyme: Speicherung, Persistenz. [US-049]
Datenschutzübersicht	Übersicht, in der Nutzer sehen können, welche Daten gespeichert werden und wo sie gespeichert sind.	Synonyme: Datenschutzansicht. Abgrenzung: Nicht Datenschutzerklärung. [US-033]
Datenschutzeinstellung	Einstellung, mit der Nutzer festlegen, wie ihre Daten verwendet, analysiert oder geschützt werden.	Synonyme: Datenschutzoption, Privatsphäre-Einstellung. [US-035]
Datentyp	Kategorie gespeicherter Daten, z.B. Stressdaten, Energiedaten, Reflexionsdaten oder Aufgaben.	Synonym: Datenkategorie. [US-033]
Datum	Kalendertag, dem ein Eintrag, eine Aufgabe, Reflexion oder Aktivität zugeordnet ist.	Abgrenzung: Datum beschreibt einen Tag; Zeitraum beschreibt eine Spanne. [US-007]
Dauer	Zeitliche Länge einer Aktivität, Phase oder eines Schlafs.	Synonym: Zeitdauer. Abgrenzung: Dauer beschreibt Länge; Zeitpunkt beschreibt Lage. [US-047]
Dauerhafte Speicherung	Speicherung, bei der Daten nach Schließen oder Neustart der Anwendung erhalten bleiben.	Synonyme: persistente Speicherung. [US-049]
Deadline	Frist, bis zu der eine Aufgabe erledigt sein soll oder muss	Synonyme: Frist, Abgabetermin. [US-024]
Detailinformationen	Zusätzliche Information zu einem ausgewählten Datenpunkt, z.B. Datum, Stresslevel und Energielevel.	Synonyme: Detaildaten, Zusatzinformationen [US-007]
Durchschnittswert	Berechneter Mittelwert aus vorhandenen Stress- oder Energiedaten für einen Zeitraum	Synonyme: Mittelwert, Durchschnitt. [US-041]
Einführungsphase	Anfangszeitraum der Nutzung, in dem Daten gesammelt werden, um eine Baseline zu ermitteln.	Synonyme: Startphase. Nicht mit Lernphase verwechseln. [US-018]
Eingabeseite	Bereich, in dem Stress- und Energielevel erfasst werden.	Abgrenzung: Nicht Analyseansicht. [US-001]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Energielevel	Subjektiver Wert, mit dem ein Studierender seine aktuelle Energie oder Leistungsfähigkeit einschätzt.	Synonyme: Energiewert, Energiedaten. Energielevel = fachlicher Zustand; Energiewert = konkreter gespeicherter Wert. [US-002]
Energiedaten	Gesamtheit erfasster Energiewerte	Synonyme: Energiewerte. Abgrenzung: Sammlung mehrerer Werte. [US-023]
Energiewert	Konkreter gespeicherter Wert eines Energielevels.	Abgrenzung: Energielevel beschreibt das Empfinden; Energiewert ist der erfasste Wert. [US-002]
Empfehlung	Allgemeiner Vorschlag des Systems, der aus analysierten Daten abgeleitet wird.	Abgrenzung: Handlungsempfehlung ist eine konkrete, umsetzbare Empfehlung. [US-023 und US-026]
Empfehlungsbeschreibung	Verständliche Beschreibung einer Empfehlung.	Abgrenzung: Beschreibung erklärt, was empfohlen wird; Erklärung erklärt, warum. [US-023]
Empfehlungsfeedback	Rückmeldung des Studierenden zu einer Empfehlung	Synonyme: Feedback zu Empfehlungen, Rückmeldungen [US-027]
Empfehlungsgrundlage	Daten und Faktoren, auf denen eine Empfehlung basiert	Synonyme: Datengrundlage, Grundlage einer Empfehlung. [US-026]
Empfehlungssystem	Systembereich, der aus Daten Empfehlungen erzeugt.	Abgrenzung: Fachlicher Systembereich, keine technische Implementierung. [US-023 bis US-026]
Erkannter Trend	Vom System identifizierte Entwicklung in Stress-, Energie- und Belastungswerten.	Abgrenzung: Ergebnis der Trend-Erkennung. [US-019]
Externe Speicherung	Speicherung von Daten außerhalb der lokalen App-Umgebung.	Synonym: extern gespeichert. Abgrenzung: Gegenteil von lokaler Speicherung. [US-033]
Exportauswahl	Auswahl der Datenkategorien und Zeiträume, die beim Export berücksichtigt werden.	Synonyme: zu exportierende Daten. [US-058]
Exportdatei	Datei, die beim Datenexport erzeugt wird und ausgewählte Daten enthält	Synonym: exportierte Datei. [US-057]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Exportierte Daten	Daten, die bereits in eine Exportdatei übertragen wurden.	Abgrenzung: Kopie gespeicherter Daten [US-059]
Exportzeitraum	Zeitraum, aus dem Daten beim Export berücksichtigt werden.	Abgrenzung: Nicht automatisch derselbe Analysezeitraum. [US-058]
Fehlende Daten	Daten, die erwartet werden aber nicht vorhanden sind.	Abgrenzung: Nicht dasselbe wie gelöschte Daten. [US-050]
Feedbackstatus	Sichtbarer Zustand, ob zu einer Empfehlung bereits Feedback abgegeben wurde.	Synonym: Rückmeldestatus. [US-027]
Filter	Auswahlkriterium, mit dem angezeigte Daten eingeschränkt werden.	Synonym: Filteroption. Abgrenzung: Filter verändert die Anzeige, nicht die gespeicherten Daten. [US-045]
Freitext-Notiz	Frei formulierter Text zu einer Reflexion oder einem besonderen Ereignis	Synonyme: Notiz, Kommentar. [US-032]
Gastnutzer	Nutzer, der die App ohne Registrierung und Benutzerkonto verwendet.	Synonym: Gast. [US-056]
Gastnutzung	Nutzung der App ohne Registrierung oder Benutzerkonto	Synonyme: Gastmodus, Nutzung ohne Account. [US-056]
Gegenmaßnahme	Handlung, mit der ein Studierender auf eine Warnung oder Belastung reagieren kann.	Synonyme: Maßnahme, Reaktion. Abgrenzung: Eine Handlungsempfehlung ist ein konkreter Vorschlag des Systems. [US-021]
Gespeicherte Daten	Alle Daten, die vom System über den Studierenden abgelegt werden.	Synonyme: gespeicherte Informationen, gespeicherte Nutzerdaten. [US-033]
Handlungsempfehlung	Konkreter Vorschlag des Systems, der Studierenden helfen soll, Verhalten oder Alltag anzupassen.	Synonyme: Empfehlung, Vorschlag. [US-023]
Hinweis	Allgemeine Benachrichtigung mit geringerer Dringlichkeit als eine Warnung.	Abgrenzung: Hinweis informiert; Warnung zeigt eine kritische Situation an. [US-011]
Individuelles Verhalten	Persönliches Nutzungs- oder Belastungsverhalten, an dem zukünftige Empfehlungen ausgerichtet werden können.	Synonym: persönliches Verhalten. [US-027]
Instabile Phase	Zeitraum, indem Werte stark schwanken oder auffällig ungleichmäßig verlaufen.	Synonyme: auffällige Phase, instabiler Zeitraum. [US-044]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Kategorie	Allgemeine Einordnung eines Elements, z.B. Aufgabe, Phase oder Benachrichtigung.	Abgrenzung: Bei Aufgabe besser "Aufgabenkategorie", bei Phasen besser "Phasenkategorie". [US-038]
Kontextdaten	Ergänzende Daten über die Situation des Studierenden, die bei Analysen berücksichtigt werden können.	Synonyme: Bewegung, Aktivität. [US-048]
Kontinuierlicher Anstieg	Entwicklung, bei der ein Wert über mehrere Tage fortlaufend steigt.	Synonyme: steigender Trend, dauerhafter Anstieg. [US-019]
Kritischer Schwellenwert	Grenzwert, ab dem ein Wert als kritisch gilt und eine Warnung ausgelöst werden kann.	Synonyme: Grenzwert, Warnschwelle. [US-020]
Kritischer Trend	Trend, der als problematisch eingestuft wird und eine Warnung auslösen kann.	Abgrenzung: Nicht jeder Trend ist kritisch. [US-019]
Lernphase	Zeitraum, in dem der Studierende lernt und der als Kontextinformationen genutzt werden kann.	Synonym: Lernzeit. [US-047]
Lokale Daten	Daten, die lokal auf dem Gerät oder in der lokalen App-Umgebung gespeichert sind.	Synonyme: lokal gespeicherte Daten. [US-034]
Lokale Nutzung	Nutzung der App, bei der Daten lokal gehalten werden	Abgrenzung: Nicht identisch mit Offline-Nutzung: lokale Nutzung kann auch ohne Benutzerkonto stattfinden. [US-056]
Messwert	Einzelner erfasster Stress- oder Energiewert	Synonyme: Wert, Datenwert. Abgrenzung: Ein Eintrag kann mehrere Messwerte enthalten. [US-006]
Monat	Vordefinierter Zeitraum zur längerfristigen Analyse.	Abgrenzung: Zeitraumoption, keine eigene Kernentität. [US-006]
Monatsansicht	Ansicht, die Daten für einen Monat darstellt.	Synonym: Monatsübersicht. [US-043]
Muster	Wiederkehrende Auffälligkeit in Reflexions-, Stress-, Energie- und Verhaltensdaten.	Synonyme: Auffälligkeit, wiederkehrende Muster. [US-030]
Nachvollziehbarkeit	Eigenschaft, dass Nutzer verstehen können, warum Warnungen oder Empfehlungen erzeugt wurden.	Synonyme: Transparenz, Erklärbarkeit. [US-022]
Niedriges Energielevel	Zustand, bei dem das Energielevel unter das normale oder persönliche Niveau fällt.	Synonyme: niedriger Energiewert, Energieabfall. [US-021]
Nutzer	Person, die die Anwendung verwendet.	Synonym: Studierender, User.

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Nutzerdaten	Daten, die vom Studierenden erfasst oder durch seine Nutzung erzeugt werden.	Synonym: Benutzerdaten. [US-041]
Onboarding	Geführter Einstieg in die App, bei dem grundlegende Informationen oder Einstellungen eingerichtet werden.	Synonyme: Einführung, Einrichtungsprozess. [US-017]
Onboarding-Fortschritt	Sichtbare Anzeige des Fortschritts innerhalb des Onboardings	Synonym: Fortschritt. [US-017]
Onboarding-Schritt	Einzelner Abschnitt innerhalb des Onboarding-Prozesses.	Abgrenzung: Teil des Onboardings. [US-017]
Persönliche Baseline	Individueller Referenzwert eines Studierenden für Stress- oder Energielevel.	Synonyme: persönlicher Normalwert, normales Niveau. [US-021]
Persönliche Informationen	Daten, die sich auf den Studierenden beziehen.	Synonym: personenbezogene Informationen. [US-033]
Phase	Allgemeiner Zeitraum mit einer bestimmten Bedeutung oder Kategorie, z.B. Lernen oder Arbeiten	Synonym: Zeitabschnitt. Lernphase und Arbeitsphase sind konkrete Ausprägungen. [US-047]
Phasenkategorie	Einordnung einer Phase, z.B. Lernen oder Arbeiten	Abgrenzung: Nicht mit Aufgabenkategorie verwechseln. [US-047]
Planungsvorschlag	Empfehlung zur Anpassung der Zeitplanung.	Synonyme: Zeitplanungsvorschlag, Vorschlag zur Planung [US-024]
Priorität	Einstufung der Wichtigkeit oder Dringlichkeit einer Aufgabe.	Synonyme: Wichtigkeit, Dringlichkeit, Abgrenzung: Nicht dasselbe wie Kategorie. [US-038]
Prioritätsstufe	Konkreter Wert einer Priorität, z.B. niedrig, mittel oder hoch.	Synonym: Prioritätswert. [US-038]
Profilinformation	Einzelne Information im Benutzerprofil	Synonyme: Profildaten. [US-055]
Reflexion	bewusste rückblickende Bewertung oder Einordnung des eigenen Tages, der eigenen Wahrnehmung oder Entwicklung.	Synonyme: Selbstreflexion, Rückblick. [US-028]
Reflexionsantwort	Antwort des Studierenden auf eine Leitfrage innerhalb einer Reflexion	Synonym: Antwort [US-031]
Reflexionsdaten	Gespeicherte Daten aus einer Reflexion	Synonyme: Reflexionseinträge

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Registrierung	Vorgang, bei dem ein Studierender ein Benutzerkonto erstellt.	Synonyme: Kontoerstellung. Abgrenzung: Registrierung ist nicht Anmeldung. [US-053]
Schlafdaten	Erfasste Informationen über den Schlaf, mindestens Schlafdauer und Datum	Synonyme: Schlafinformationen, Schlafdaten sind Kontextdaten. [US-046]
Schlafdauer	Dauer des Schlafs für einen bestimmten Tag oder Zeitraum	Synonym: Schlafzeit. [US-046]
Schwankung	Auffällige Veränderung von Werten innerhalb eines Zeitraums.	Synonyme: Wertschwankung, Fluktuation. [US-044]
Sensible Daten	Besonders schützenswerte Daten, bei deren Verarbeitung oder Weitergabe besondere Vorsicht nötig ist.	Synonyme: sensible Informationen. [US-059]
Sensible Nutzerdaten	Persönliche oder schützenswerte Nutzerdaten des Studierenden.	Synonyme: sensible Daten, schützenswerte Daten. [US-052]
Serverseitige Daten	Daten, die außerhalb des Geräts auf einem Server gespeichert sind.	Synonyme: Serverdaten, extern gespeicherte Daten. [US-034]
Sitzung	Zeitraum, in dem ein angemeldeter Nutzer die App aktiv verwendet.	Synonym: Session, Nutzungssitzung. [US-054].
Skalenwert	Einzelner auswählbarer Wert innerhalb einer Stress- oder Energieskala.	Abgrenzung: Skalenwert = konkreter Wert; Skala = Bewertungssystem. [US-001]
Speicherdauer	Zeitraum, für den Daten gespeichert bleiben.	Abgrenzung: In den User Storys eher implizit durch gespeicherte Daten und Historie relevant. [US-014]
Speicherort	Ort oder Bereich, an dem Daten gespeichert werden, z.B. lokal oder extern.	Abgrenzung: Fachlicher Begriff für Transparenz, keine technische Speicherbeschreibung. [US-033]
Stimmungsauswahl	Auswahlmöglichkeit, mit der der Studierende seine Stimmung dokumentiert.	Synonyme: Stimmung, Mood-Auswahl [US-028]
Stressdaten	Gesamtheit erfasster Stresswerte	Synonyme: Stresserte [US-039]
Stresslevel	Subjektiver Wert, mit dem der Studierende seinen aktuellen Stresszustand einschätzt.	Synonyme: Stresswert, Stressdaten, Stresslevel = fachlicher Zustand; Stresswert = konkreter gespeicherter Wert. [US-001]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Stressskala	Skala zur Auswahl eines Stresslevels.	Abgrenzung: Stressskala ist das Bewertungssystem; Stresslevel ist der ausgewählte Zustand. [US-001]
Stressursache	Möglicher Auslöser oder Einflussfaktor, der zu erhöhtem Stress beiträgt.	Synonyme: Ursache für Stress, Belastungstreiber. [US-025]
Stressverlauf	Zeitliche Entwicklung gespeicherter Stresswerte	Abgrenzung: Betrachtet Stresswerte, nicht Energielevel. [US-005]
Stresswert	Konkreter gespeicherter Wert eines Stresslevels.	Abgrenzung: Stresslevel = subjektiver Zustand; Stresswert = gespeicherter Zahlen-/Skalenwert. [US-001]
Studierender	Primärer Nutzer der Anwendung, der Daten erfasst, reflektiert und auswertet.	Synonym: Nutzer, User. [In den User Storys: "Als Studierender"]
Subjektive Wahrnehmung	Persönliche Einschätzung des Studierenden zu Tag, Belastung oder Wohlbefinden	Synonyme: persönliche Wahrnehmung, Selbsteinschätzung [US-028]
Synchronisation	Abgleich lokal gespeicherter Daten mit extern gespeicherten Daten.	Synonym: Datenabgleich, Sync. [US-051]
Tagesbewertung	Rückblickende Bewertung eines abgeschlossenen Tages.	Synonyme: Tagesreflexion, Tag bewerten. [US-028]
Tag	Kleinster vordefinierter Zeitraum zur Analyse oder Reflexion	Abgrenzung: Ein Tag ist ein Zeitraum, nicht automatisch ein Datenpunkt. [US-006]
Tagesansicht	Ansicht, die Daten für einen Tag darstellt.	Synonym: Tagesübersicht. [US-043]
Tagesverlauf	Darstellung gespeicherter Werte eines Tages	Synonym: Verlauf. [US-001]
Teilen-Bestätigung	Sicherheitsabfrage vor dem Teilen sensibler Daten.	Synonyme: Freigabebestätigung, Bestätigung vor Datenfreigabe. [US-059]
Termin	Zeitlich festgelegte Verpflichtung oder Ereignis	Synonyme: Ereignis, Kalendetermin. Abgrenzung: Eine Aufgabe muss nicht zwingend ein Termin sein. [US-037]
Transparenz	Nachvollziehbarkeit darüber, welche Daten gespeichert oder verwendet werden.	Synonym: Einsicht, Nachvollziehbarkeit. [US-033]
Trend	Erkennbare Entwicklung von Werten über mehrere Tage, z.B. steigend, fallend oder stabil.	Synonyme: Entwicklung, Tendenz. [US-008]

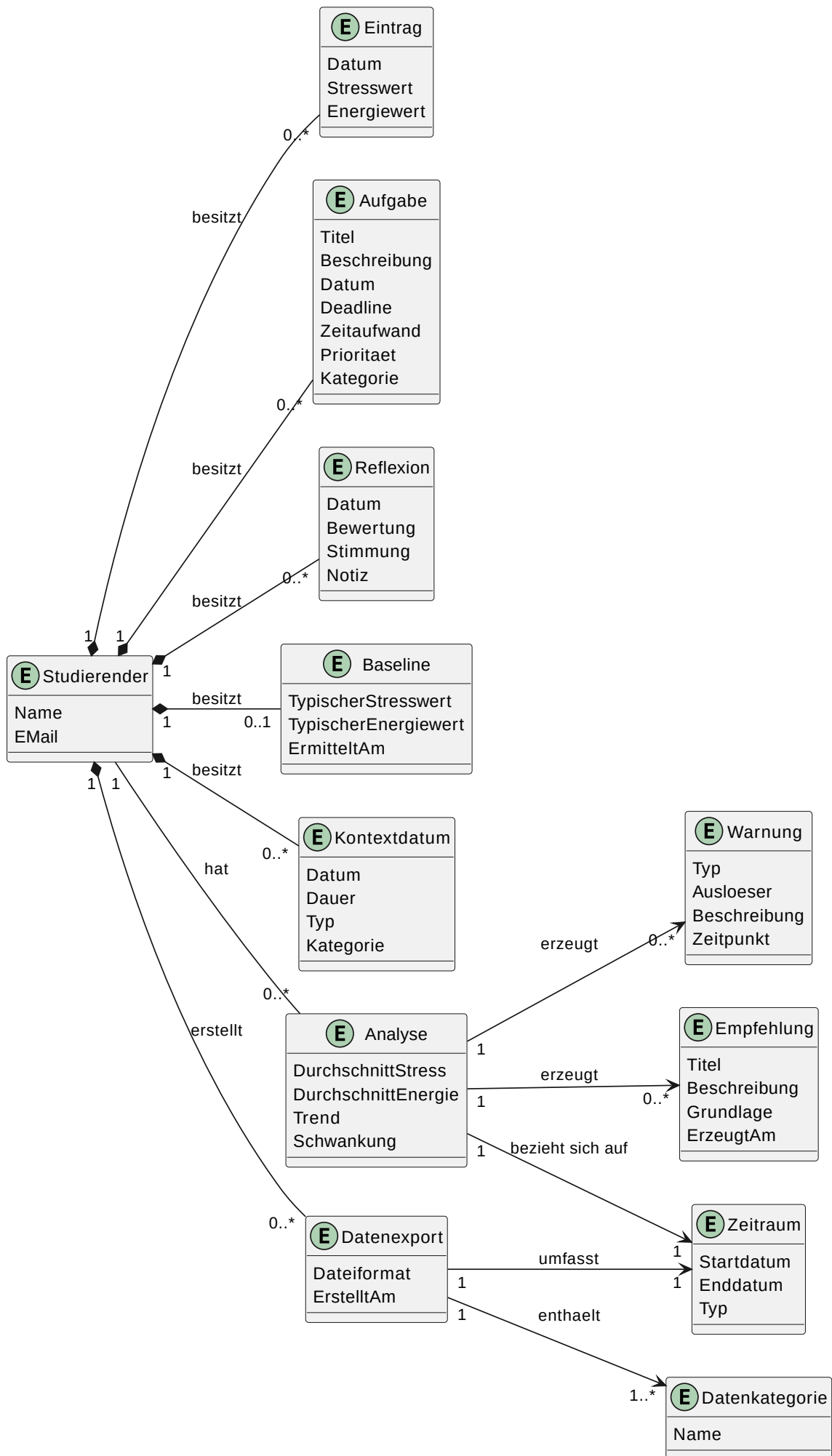
Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Trend-Erkennung	Fachliche Logik, mit der erkannt wird, ob Werte fallen, steigen oder stabil bleiben.	Synonyme: Trenderkennung, Trendanalyse. [US-019]
Trendinformation	Angezeigte Information darüber, ob sich Werte steigend, fallend oder stabil entwickeln.	Abgrenzung: Ergebnis der Trend-Erkennung. [US-008]
Typisches Energielevel	Persönlicher Normalwert des Energielevels.	Synonyme: Energie-Baseline, persönlicher Energiewert. [US-016]
Typisches Stresslevel	Persönlicher Normalwert des Stresslevels.	Synonyme: Stress-Baseline, persönlicher Stresswert. [US-016]
Unberechtigter Zugriff	Zugriff auf Daten ohne Erlaubnis oder Berechtigung.	Synonyme: unbefugter Zugriff. [US-052]
Vergleich	Gegenüberstellung von Daten, z.B. Reflexionsdaten mit Messwerten.	Synonyme: Abgleich, Gegenüberstellung. [US-029]
Vergleichsansicht	Ansicht, in der mehrere Zeiträume oder Werte gegenübergestellt werden.	Synonym: Vergleichsübersicht. [US-042]
Vergleichswert	Wert, der zum Vergleich mit einer Baseline oder einem anderen Zeitraum verwendet wird.	Synonyme: Referenzwert, Vergleichsdaten. [US-042]
Verknüpfung	Fachliche Zuordnung zwischen zwei Datenbereichen, z.B. Aufgabe und Stresswert.	Synonyme: Zuordnung, Verbindung. [US-039]
Verschlüsselte Speicherung	Speicherung sensibler Daten in geschützter Form.	Synonym: geschützte Speicherung. Abgrenzung: Fachlich relevant wegen Datenschutz, ohne technische Details auszuführen. [US-052]
Vollständige Löschung	Entfernen aller zu einem Nutzer gehörenden Daten.	Synonyme: Datenlöschung, endgültige Löschung. [US-034]
Warnung	Kritische Benachrichtigung, die auf eine problematische Situation aufmerksam macht.	Abgrenzung: Warnung ist stärker als Hinweis. [US-019]
Warnungserklärung	Verständliche Begründung, warum eine Warnung erzeugt wurde.	Synonyme: Erklärung, Begründung der Warnung. [US-022]
Warnungsregel	Fachliche Regel, nach der entschieden wird, warum eine Warnung erzeugt wurde.	Synonyme: Erklärung, Begründung der Warnung. [US-022]
Woche	Vordefinierter Zeitraum, typischerweise sieben Tage.	Abgrenzung: Analysezeitraum, nicht automatisch Kalenderwoche. [US-006]
Wochenansicht	Ansicht, die Daten oder Reflexionen über eine Woche zusammenfasst.	Synonym: Wochenübersicht. [US-030]

Begriff / Abkürzung	Definition	Synonyme / Abgrenzungen
Wochenreflexion	Rückblickende Betrachtung einer Woche, um Entwicklungen, Trends oder Muster zu erkennen.	Synonym: Woche reflektieren. [US-030]
Wohlbefinden	Subjektiver Zustand des Studierenden, beeinflusst durch Stress, Energie, Schlaf, Aktivität und weitere Faktoren	Synonym: Befinden. Abgrenzung: Breiter als Stresslevel oder Energielevel. [US-048]
Zeitaufwand	Geschätzte oder erfasst Zeit, die für eine Aufgabe benötigt wird.	Synonym: Aufwand, Bearbeitungszeit. [US-040]
Zeitliche Zuordnung	Zuordnung von Daten anhand ihrer zeitlichen Nähe oder desselben Zeitraums	Synonym: zeitbezogene Verknüpfung. [US-039]
Zeitplanung	Planung von Aufgaben, Deadlines und Belastung über einen Zeitraum hinweg.	Synonyme: Tagesplanung, Aufgabenplanung. [US-024]
Zeitraum	Ausgewählter Analysebereich, für den Daten angezeigt oder ausgewertet werden.	Synonyme: Zeitspanne, Analysezeitraum. [US-006]
Zeitraumfilter	Filter, der Daten nach einem ausgewählten Zeitraum einschränkt.	Synonyme: Zeitfilter, Filter nach Zeitraum. [US-045]
Zeitraumvergleich	Vergleich von Entwicklungen oder Werten über verschiedene Zeiträume hinweg.	Synonyme: Vergleich über Zeiträume, Periodenvergleich. [US-043]
Zeitpunkt	Konkrete zeitliche Lage eines Ereignisses, einer Phase oder eines Eintrags	Synonyme: Startzeit, Zeitangabe. Abgrenzung: Zeitpunkt ist nicht Dauer. [US-047]
Zugriff	Möglichkeit, gespeicherte Daten einzusehen oder zu verwenden.	Synonym: Datenzugriff. [US-054]
Zugangsdaten	Daten, mit denen sich ein Nutzer anmeldet, typischerweise E-Mail-Adresse und Passwort.	Synonyme: Login-Daten, Anmeldedaten. [US-054]
Zusammenhang	Fachlich erkannte oder dargestellte Beziehung zwischen Daten, z.B. Stress, Energie, Aufgaben und Kontext.	Synonyme: Beziehung, Korrelation. "Korrelation" nur nutzen, wenn statistisch gemeint. [US-009]

3.2. Domänenmodell

Das Domänenmodell zeigt die wichtigsten fachlichen Konzepte (Entitäten) und ihre Beziehungen. Es beschreibt die Realität des Problemraums, nicht die Datenbankstruktur.

3.2.1. Diagramm





Das Domänenmodell wird im Laufe des Projekts weiterentwickelt. Änderungen sollten immer gemeinsam im Team besprochen und hier nachgepflegt werden.

3.2.2. Beschreibung der Entitäten

Kurze Erläuterung zu den im Modell enthaltenen Entitäten.

Entität	Beschreibung
Studierender	Zentrale Person im Domänenmodell. Der Studierende erfasst Stress- und Energiewerte, Aufgaben, Reflexionen und Kontextdaten. Alle persönlichen Daten und Auswertungen werden ihm fachlich zugeordnet.
Eintrag	Dokumentiert Stress- und Energiewerte zu einem bestimmten Datum. Einträge bilden die Grundlage für Verläufe, Analysen, Warnungen und Empfehlungen.
Aufgabe	Beschreibt eine Verpflichtung oder Tätigkeit des Studierenden. Aufgaben können mit Datum, Deadline, Zeitaufwand, Priorität und Kategorie beschrieben werden und helfen, Belastungstreiber zu erkennen.
Reflexion	Subjektive Bewertung eines Tages oder Zeitraums durch den Studierenden. Diese kann Bewertung, Stimmung und Notiz enthalten und ergänzt die gemessenen Stress- und Energiewerte um persönliche Wahrnehmung.
Baseline	Persönlicher Referenzwert für typische Stress- und Energiewerte eines Studierenden. Diese dient als Grundlage, um Abweichungen und auffällige Entwicklungen zu erkennen.
Kontextdaten	Ergänzende Information über Einflussfaktoren wie Schlaf, körperliche Aktivität oder Lern- und Arbeitsphasen. Kontextdaten helfen, Stress- und Energieverläufe besser zu interpretieren.
Analyse	Fachliche Auswertung vorhandener Daten. Eine Analyse kann Durchschnittswerte, Trends und Schwankungen ermitteln und bildet die Grundlage für Warnungen und Empfehlungen.
Warnung	Kritischer Hinweis an den Studierenden, wenn auffällige oder problematische Entwicklungen erkannt werden, zum Beispiel steigendes Stresslevel oder ein niedriges Energielevel.
Empfehlung	Vorschlag zur Verbesserung von Verhalten, Zeitplanung oder Belastungssteuerung. Empfehlungen basieren auf analysierten Daten und sollen den Studierenden bei der Selbststeuerung unterstützen.
Zeitraum	Zeitlicher Bereich, auf den sich eine Analyse oder ein Datenexport bezieht. Ein Zeitraum kann zum Beispiel Tag, Woche oder Monat sein.
Datenexport	Fachliches Konzept zum Export ausgewählter Daten. Ein Datenexport bezieht sich auf einen Zeitraum und enthält bestimmte Datenkategorien.
Datenkategorie	Gruppe von Daten, die bei einem Export ausgewählt werden kann, zum Beispiel Stressdaten, Energiedaten, Aufgaben oder Reflexionen.

3.2.3. Beschreibung der Beziehungen

Beschreibung der wichtigsten Beziehungen zwischen den Entitäten, sofern diese besondere Kardinalitäten haben oder aus anderen Gründen erklärungsbedürftig sind.

Von	Zu	Kardinalität	Bedeutung
Studierender	Eintrag	1 zu 0..*	Ein Studierender kann keine, einen oder viele Einträge besitzen. Jeder Eintrag gehört genau zu einem Studierenden.
Studierender	Aufgabe	1 zu 0..*	Ein Studierender kann mehrere Aufgaben verwalten. Jede Aufgabe ist genau einem Studierenden zugeordnet.
Studierender	Reflexion	1 zu 0..*	Ein Studierender kann mehrere Reflexionen erfassen. Jede Reflexion gehört genau zu einem Studierenden.
Studierender	Baseline	1 zu 0..1	Ein Studierender kann keine oder eine aktuelle Baseline besitzen. Die Baseline ist optional, da diese erst nach einer Einführungsphase oder initialen Festlegung vorhanden sein kann.
Studierender	Kontextdaten	1 zu 0..*	Ein Studierender kann mehrere Analysen besitzen oder durchführen lassen. Jede Analyse gehört genau zu einem Studierenden.
Studierender	Analyse	1 zu 0..*	ein Studierender kann mehrere Analysen besitzen oder durchführen lassen. Jede Analyse gehört zu genau einem Studierenden.
Analyse	Warnung	1 zu 0..*	Eine Analyse kann keine, eine oder mehrere Warnung erzeugen. Warnungen entstehen auf Grundlage ausgewerteter Daten.
Analyse	Empfehlung	1 zu 0..*	Eine Analyse kann keine, eine oder mehrere Empfehlungen erzeugen. Empfehlungen entstehen auf Grundlage ausgewerteter Daten.
Analyse	Zeitraum	1 zu 1	Jede Analyse bezieht sich auf genau einen Zeitraum. Dadurch ist klar, für welchen zeitlichen Bereich Werte, Trends oder Schwankungen betrachtet werden.
Studierender	Datenexport	1 zu 0..*	Ein Studierender kann mehrere Datenexporte erstellen. Jeder Export gehört zu genau einem Studierenden.
Datenexport	Zeitraum	1 zu 1	Jeder Datenexport bezieht sich auf genau einen Zeitraum, aus dem Daten exportiert werden.
Datenexport	Datenkategorie	1 zu 1..*	Ein Datenexport enthält mindestens eine Datenkategorie. Dadurch kann der Studierende gezielt auswählen, welche Daten exportiert werden.

Dieses Dokument wird kontinuierlich gepflegt. Änderungen an Fachbegriffen oder Beziehungen müssen hier nachgezogen werden, bevor sie in User Stories oder Code einfließen.

4. UX-Konzept: F2 - EnergyBalance

Dieses Dokument beschreibt, wer das Produkt nutzt und wie — aus der Perspektive der Nutzenden. Es ergänzt das Produktkonzept um Personas, Nutzungskontext und systemweite Interaktionsflüsse und dient als gemeinsame Referenz für Design- und Entwicklungsentscheidungen.

Individuelle Wireframes einzelner Features werden direkt in den zugehörigen User Stories im Product Backlog gepflegt (verlinkt oder kopiert aus Miro oder anderem Tool). Dieses Dokument enthält nur Artefakte, die mehrere Stories oder das gesamte Produkt betreffen.

4.1. Personas

Personas sind fiktive, aber realitätsbasierte Beschreibungen typischer Nutzender. Sie helfen dem Team, Entscheidungen konsequent aus Nutzerperspektive zu treffen. Personas sollten auf echten Nutzerinterviews oder Beobachtungen basieren — nicht auf Annahmen allein.

Hinweis: Im [Produktkonzept](#) werden Zielgruppen nur benannt. Hier werden sie als Personas ausgearbeitet.

4.1.1. Persona 1: Max, Mustermann

Rolle	Studierende im 4. Semester, Wirtschaftsinformatik
Alter	22 Jahre
Zitat	„Ich merke oft erst zu spät, dass alles zu viel wird.“
Ziele	Max will seine Aufgaben besser planen um Stress frühzeitig zu erkennen und zu vermeiden.
Frustrationen	Er nutzt Kalender und To-do-Listen, sieht dort aber nicht, wie belastend einzelne Aufgaben für ihn wirklich sind.
Verhalten & Kontext	Er nutzt hauptsächlich sein Smartphone zwischen Vorlesungen, in der Bahn oder abends zu Hause. Eingaben müssen schnell und einfach sein.
Bezug zum Produkt	EnergyBalance hilft Max, täglich Energielevel und Stress zu erfassen und diese Werte mit Aufgaben zu verknüpfen.

4.1.2. Persona 2: Maja, Schneider

Rolle	Studienanfängerin im 1. Semester
Alter	18 Jahre
Zitat	„Ich weiß oft nicht, ob ich zu wenig mache oder mich schon überlaste.“
Ziele	Sie möchte eine gesunde Lernroutine aufbauen, ihre Arbeitsbelastung realistischer einschätzen und sich im Studium besser organisieren können.

Frustrationen	Sie hat Schwierigkeiten, ihren Lernaufwand realistisch einzuschätzen und sich im Studium effektiv selbst zu organisieren.
Verhalten & Kontext	Maja nutzt digitale Tools täglich und arbeitet hauptsächlich mobil am Smartphone oder Tablet. Sie benötigt klare Visualisierungen und einfache Funktionen.
Bezug zum Produkt	EnergyBalance unterstützt sie dabei, ihre Belastung besser einzuschätzen, Lernphasen zu planen und gesunde Gewohnheiten aufzubauen.

4.1.3. Persona 3: Thai Minh, Nguyen

Rolle	Student im 2.Semester
Alter	20 Jahre
Zitat	<i>„Es gibt nur zwei Arten von Menschen. Die Macher und die Lacher.“</i>
Ziele	Er will seinen bereits bestehenden Lernplan maximieren, um sein Studium auf eine neue Ebene zu heben.
Frustrationen	Er möchte seine humanen Schwankungen, die sich in seinen Noten widerspiegeln, auslöschen.
Verhalten & Kontext	Thai Minh nutzt digitale Tools täglich und arbeitet hauptsächlich am PC oder Smartphone. Funktionen sollten eindeutig und selbsterklärend sein.
Bezug zum Produkt	EnergyBalance assistiert ihn dabei, Lernphasen und freie Tage für neue Kompetenzen zu planen.



Für jedes Projekt sind 2–4 Personas ausreichend. Mehr Personas verwässern den Fokus. Wenn eine Persona in keiner User Story vorkommt, wird sie gestrichen.

4.2. Nutzungskontext

Der Nutzungskontext gemäß DIN EN ISO 9241-11 beschreibt die Lebenswirklichkeit der Personas — unabhängig vom Produkt. Er zeigt, in welchem Umfeld, mit welchen Mitteln und unter welchen Bedingungen das Produkt genutzt wird.

Aspekt	Beschreibung
Benutzer	Hauptsächlich Studierende mit unterschiedlicher Erfahrung in Selbstorganisation.
Ziele & Aufgaben	Energielevel erfassen, Stress dokumentieren, Aufgaben planen, Muster erkennen, Überlastung vermeiden.
Ressourcen	Smartphone, Laptop, Web-App, Internetzugang, Hochschulaccount oder eigenes Login.

Aspekt	Beschreibung
Umgebung	Technische Umgebung: Mobile App als Hauptplattform, ergänzend Web-App.
	Physische Umgebung: Nutzung unterwegs, in der Hochschule, zu Hause oder während Lernphasen.
	Soziale & organisatorische Umgebung: Studium, Prüfungsstress, Nebenjob, Gruppenarbeiten, Beratungssituationen.

4.2.1. Aktuelle Situation (ohne das Produkt)

Wie lösen die Nutzenden ihr Problem heute? Diese Beschreibung zeigt, was das Produkt verbessern soll — und macht den Mehrwert explizit.

Studierende organisieren ihren Studienalltag häufig mit Kalendern, To-do-Listen oder Notizen. Das persönliche Stress- und Energielevel wird dabei meist nur subjektiv eingeschätzt und nicht systematisch dokumentiert. Dadurch bleiben Zusammenhänge zwischen Arbeitsbelastung und Wohlbefinden oft unentdeckt.

4.2.2. Nutzungshürden & Risiken

Was könnte die Akzeptanz oder Nutzung des Produkts beeinträchtigen?

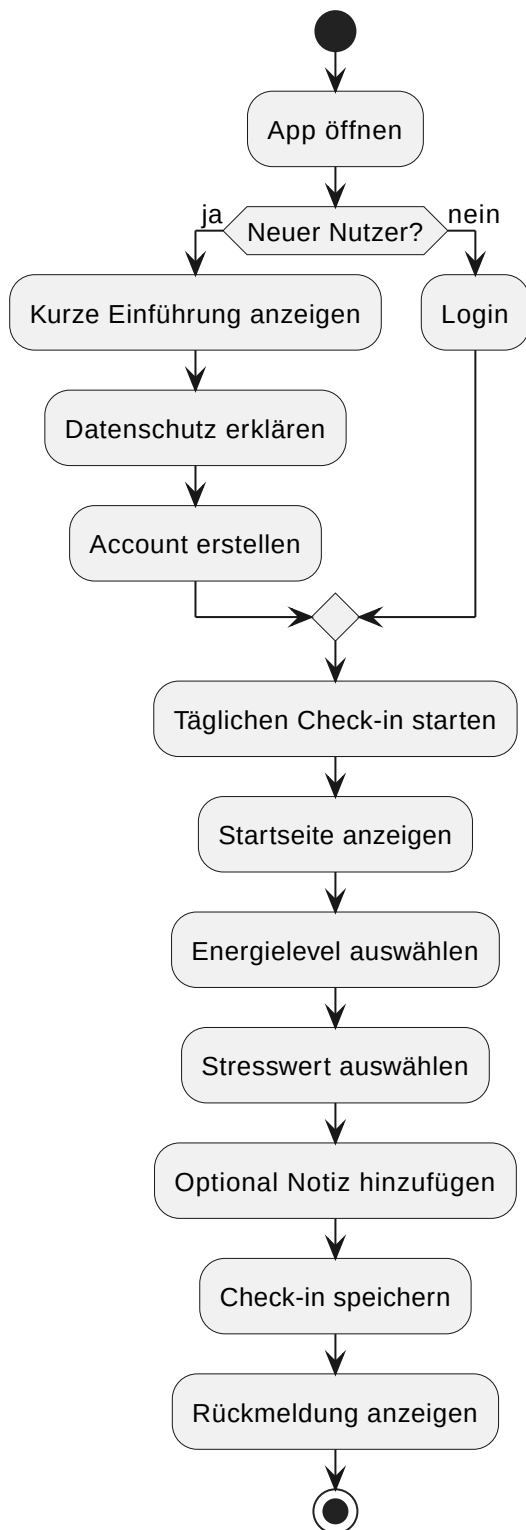
Hürde / Risiko	Mögliche Reaktion im Design
Datenschutzbedenken bei sensiblen Daten	Transparente Datenschutzhinweise und vollständige Kontrolle über Datenfreigaben
Zu hoher Eingabeaufwand	Check-ins in weniger als einer Minute
Gefahr der Überforderung durch zu viele Funktionen	Klare Struktur und Fokus auf Kernfunktionen
Fehlende Motivation zur langfristigen Nutzung	Erinnerungen, Wochenrückblicke und sichtbarer Mehrwert
Unverständliche Diagramme	Einfache Visualisierung mit kurzen Erklärungen
Unterschiedliche Stressfaktoren zwischen Nutzenden	Hohe Personalisierbarkeit der Eingaben

4.3. Systemweite Interaktionsflüsse

Dieser Abschnitt beschreibt Abläufe, die mehrere Features oder Bereiche des Produkts umfassen. Einzelne Feature-Wireframes werden in den jeweiligen User Stories gepflegt. Hier dokumentieren wir nur Flüsse auf Systemebene, z.B. mit UML-Aktivitätsdiagrammen.

4.3.1. Flow 1: Onboarding & erster Check-in

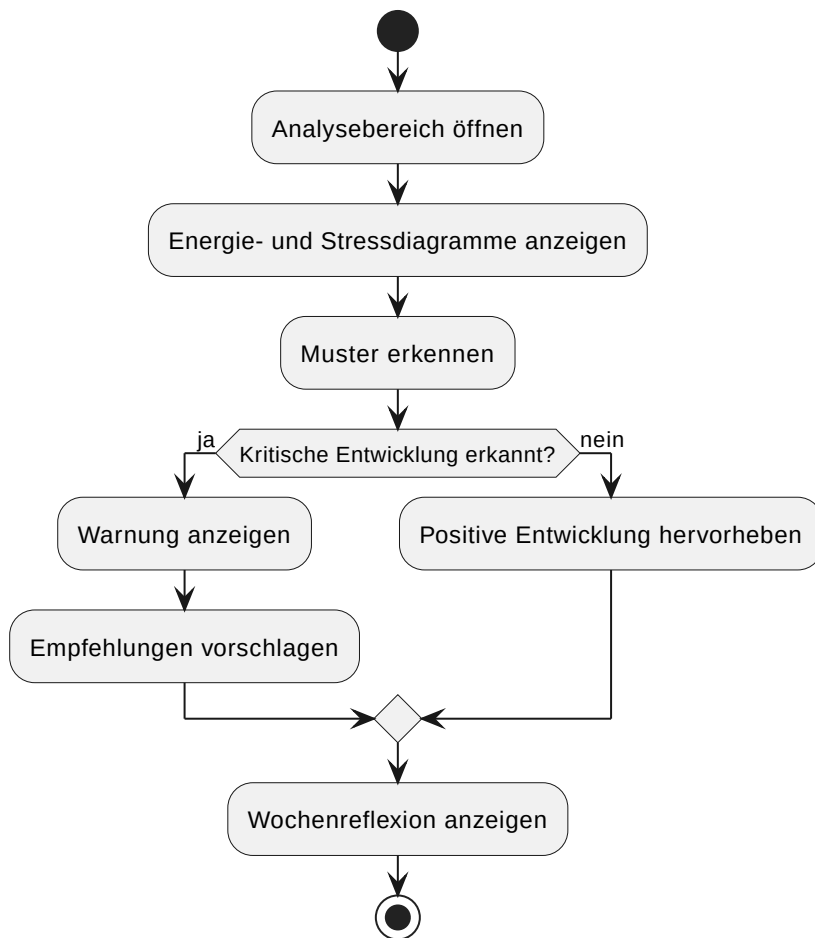
Dieser Flow beschreibt den Einstieg neuer Nutzer:innen sowie die erste Erfassung von Energie und



Schritt	Beteiligte Persona	Anmerkung
App öffnen	Max (Student)	Der Einstieg muss sofort verständlich sein
Datenschutz erklären	Max	Sensible Daten benötigen Vertrauen
Check-in	Max	Kernfunktion der Anwendung
Rückmeldung	Max	Direkter Nutzen motiviert zur Nutzung

4.3.2. Flow 2: Aufgaben mit Belastung verknüpfen

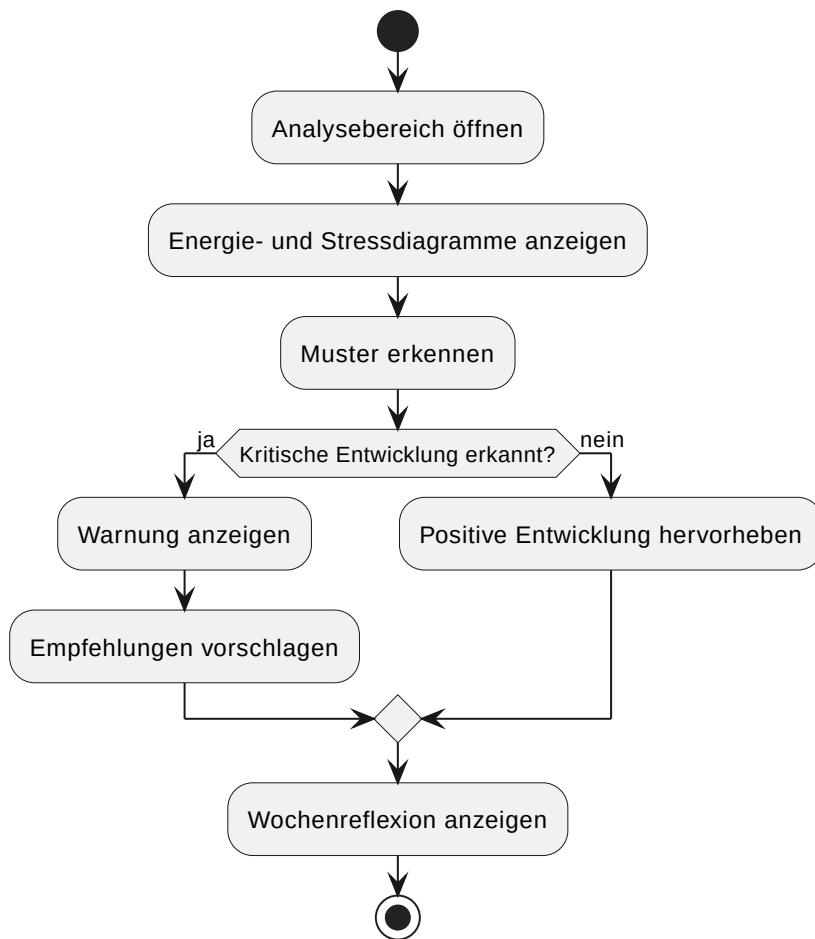
Dieser Flow beschreibt, wie Aufgaben mit Stress und Energie verbunden werden.



Schritt	Beteiligte Persona	Anmerkung
Aufgabe erstellen	Max (Student)	Vergleichbar mit To-do-App
Belastung einschätzen	Max	Verknüpfung zwischen Aufgaben und Stress
Warnhinweise	Max	Frühzeitige Prävention von Überlastung

4.3.3. Flow 3: Analyse & Empfehlungen

Dieser Flow zeigt die Analysefunktionen und personalisierte Empfehlungen.



Schritt	Beteiligte Persona	Anmerkung
Diagramme anzeigen	Max (Student)	Langfristige Muster sichtbar machen
Warnungen	Max	Frühwarnsystem bei Überlastung
Empfehlungen	Max	Handlungsorientierter Mehrwert
Reflexion	Max	Unterstützung der Selbstreflexion

4.4. Designsystem & Interaktionsprinzipien

Dieser Abschnitt dokumentiert systemweite Design- und Interaktionsentscheidungen, die konsistent über alle Features hinweg gelten. Sie bilden die Basis für alle Wireframes und UI-Implementierungen.

4.4.1. Visuelle Sprache

Aspekt	Entscheidung & Begründung
Farbpalette	Ruhige Blau- und Grüntöne vermitteln Balance und Vertrauen
Akzentfarben	Warme Farben für Warnungen und Hinweise
Typografie	Gut lesbare serifenlose Schrift mit Fokus auf mobile Nutzung
Ikongrafie	Klare und einfache Icons für Energie, Stress, Aufgaben und Analyse

Aspekt	Entscheidung & Begründung
Bildsprache	Minimalistisch und beruhigend statt überfordernd

4.4.2. Interaktionsprinzipien

Vereinbarte UX-Regeln, die für das gesamte Produkt gelten. Jede Entscheidung wird begründet, damit sie nicht aus Unwissenheit verletzt wird.

Prinzip	Beschreibung & Begründung
Einfachheit zuerst	Alle Kernfunktionen müssen intuitiv und schnell erreichbar sein.
Tracking darf nicht anstrengend sein	Tägliche Eingaben dauern maximal eine Minute.
Personalisierung ist zentral	Kategorien, Farben und Eingabemethoden anpassbar.
Empfehlungen statt nur Daten	Die App gibt konkrete und verständliche Handlungsvorschläge.
Positive Kommunikation	Hinweise unterstützen statt Druck aufzubauen.
Datenkontrolle bleibt beim Nutzer	Freigaben sind freiwillig und transparent.
Mobile-First	Hauptnutzung erfolgt auf dem Smartphone.

4.4.3. Barrierefreiheit

Aspekt	Anforderung
Kontrastverhältnis	Mindestens WCAG 2.1 Level AA (Kontrast $\geq 4,5:1$ für Fließtext)
Tastaturnavigation	Alle Kernfunktionen sind per Tastatur bedienbar
Screenreader	Alle interaktiven Elemente haben sinnvolle ARIA-Labels
Schriftgröße	Keine Schrift unter 14px; Nutzer:innen können Systemschriftgröße nutzen
Diagramme	Informationen zusätzlich textlich erklären
Sprache	Verständliche und nicht-technische Formulierungen

Dieses Dokument wird kontinuierlich weiterentwickelt — insbesondere nach Nutzerinterviews, Usability-Tests oder wesentlichen Designentscheidungen. Individuelle Feature-Wireframes werden nicht hier, sondern direkt in den User Stories im Product Backlog gepflegt.

Der Zweck dieses Dokuments ist es, die wesentlichen Bedarfe und Funktionalitäten des Systems Software-Name überblicksartig zu beschreiben. Der Fokus liegt auf den Fähigkeiten, die von Stakeholdern und adressierten Nutzern benötigt werden, und der Begründung dieser Bedarfe. Die Details, wie das System Software-Name diese Bedarfe erfüllt, werden durch User Stories im Product Backlog sowie dem UX-Konzept beschrieben.

Technische Spezifikation

- Architekturdokument
- Softwaredesign
- Testkonzept
- Deployment & Betrieb

5. Architekturdokument: F2 - EnergyBalance

*Dieses Dokument beschreibt die technische Architektur des Systems — welche Technologien eingesetzt werden, wie das System in seine Umgebung eingebettet ist, und welche Architekturentscheidungen getroffen wurden. Der Fokus liegt auf dem **Warum** hinter technischen Entscheidungen.*

Qualitätsziele und Einschränkungen sind im Produktkonzept als einziger Quelle definiert. Dieses Dokument referenziert sie und dokumentiert ausschließlich die technischen Konsequenzen, die sich daraus ergeben.

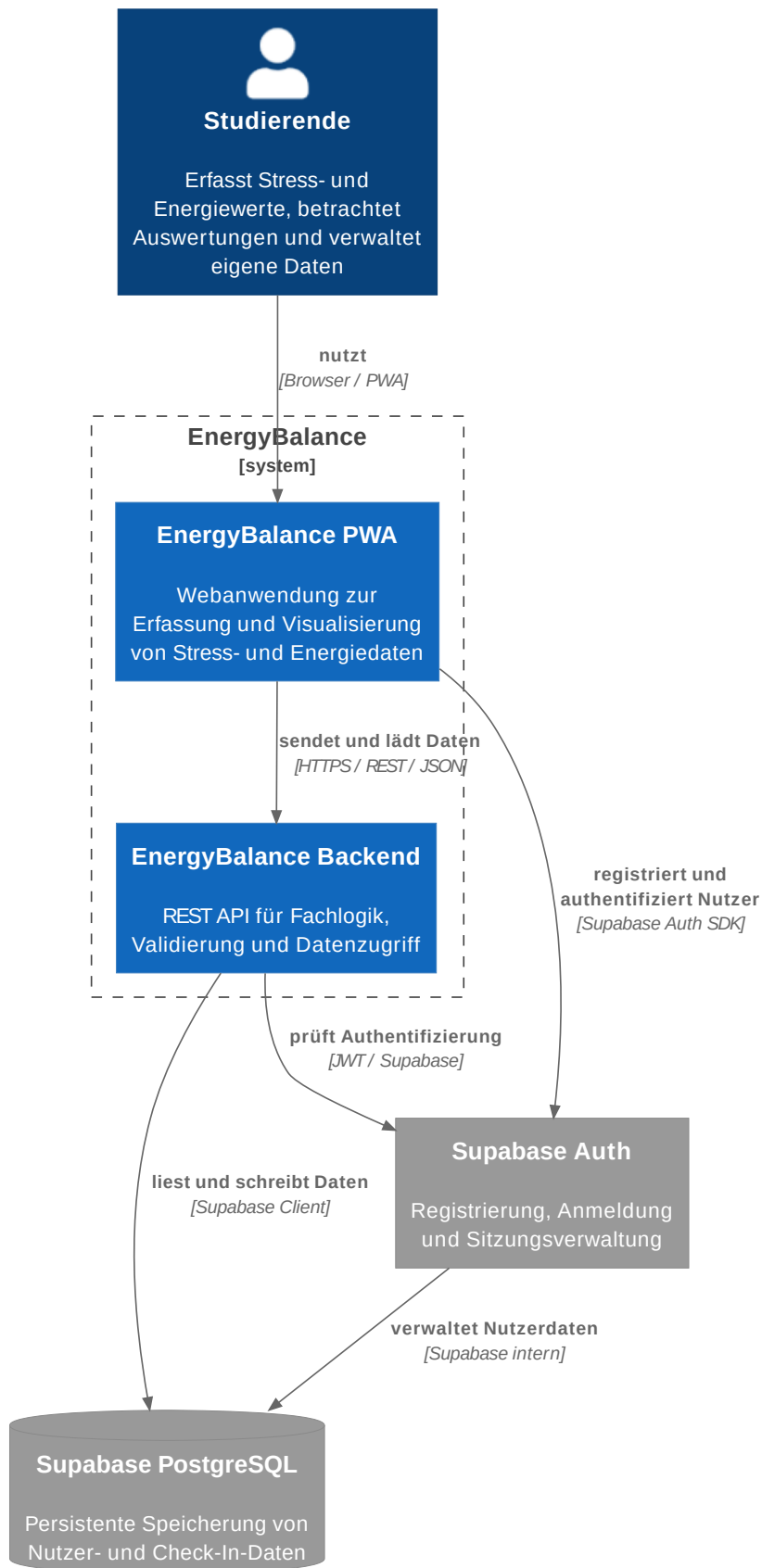
Detaillierte Beschreibungen der internen Softwarestruktur, z.B. Schichten, Komponenten, Datenmodell, Interaktionsflüsse, sind im [Designokument](#) (`design.adoc`) beschrieben.

5.1. Architektursichten

Der Systemkontext zeigt das System als Ganzes in seiner Umgebung: welche externen Akteure und Systeme mit ihm interagieren, und über welche Schnittstellen. Er beantwortet die Frage: Was gehört zum System, und was liegt außerhalb?

5.1.1. Systemkontext

System Context Diagram - EnergyBalance



Legend

person
system
external person
external system



Das Kontextdiagramm zeigt ausschließlich externe Schnittstellen. Die interne Struktur des Systems ist im Designdokument beschrieben.

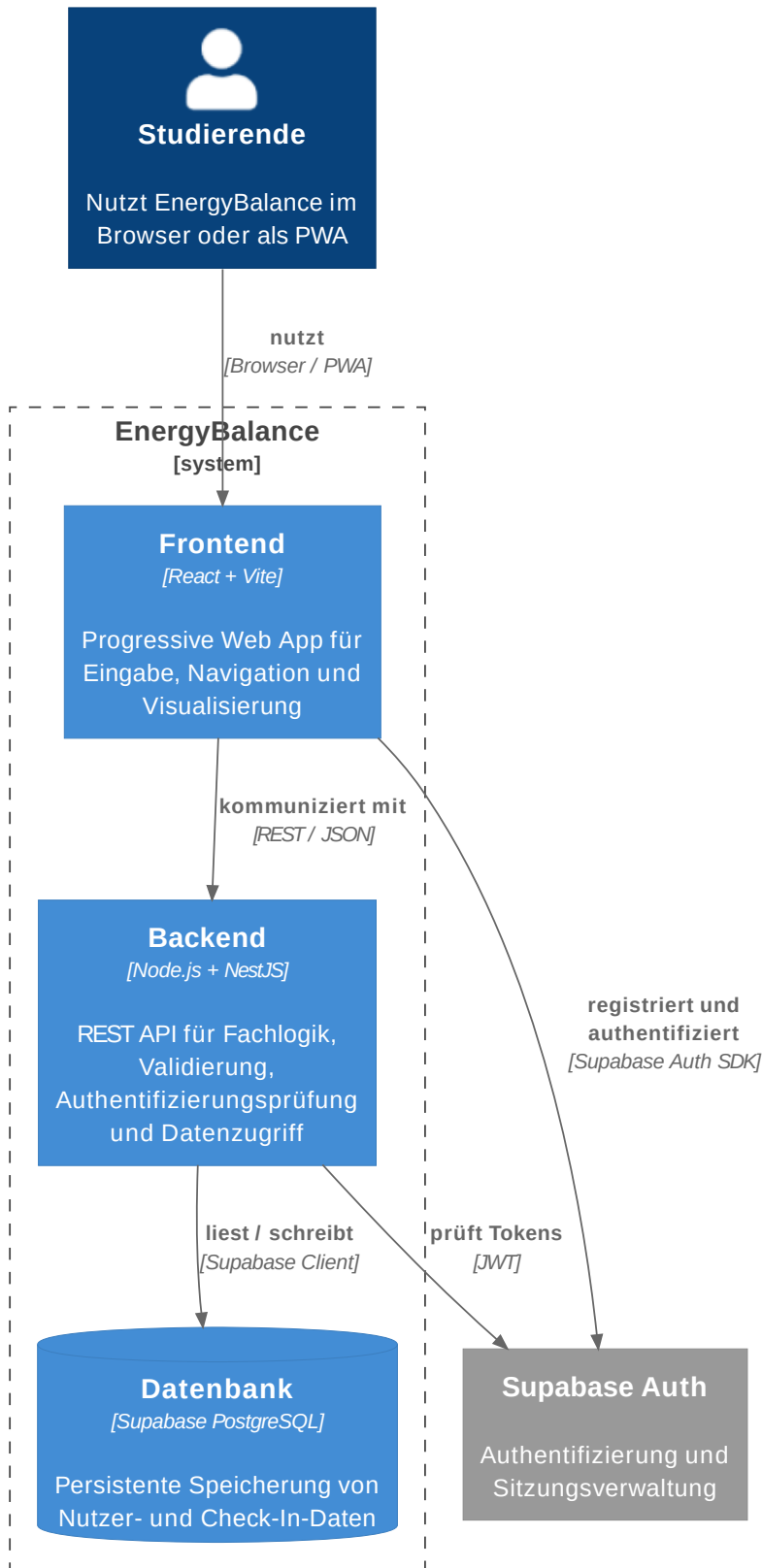
Externe Schnittstellen:

Schnittstelle	Typ	Beschreibung	Protokoll / Format
Studierende über Browser / PWA	Menschlicher Akteur	Nutzung der Anwendung zur Eingabe von Stress- und Energiewerten sowie zur Anzeige von Auswertungen	HTTPS / Web UI
Supabase Auth	Externer Dienst	Registrierung, Anmeldung und Sitzungsverwaltung	HTTPS / Supabase Auth SDK / JWT
Supabase PostgreSQL	Extern verwaltete Datenbank	Persistente Speicherung von Nutzer- und Check-In-Daten	Supabase Client / PostgreSQL
GitHub	Entwicklungsplattform	Versionsverwaltung, Issues, Pull Requests und Dokumentation	Git / HTTPS

5.1.2. Laufzeitumgebungen (Container Diagramm)

Diese Sicht gibt an, welche Umgebungen für den Code vorgesehen sind, z.B. JVM, Node.js, DBMS, Browser, Android Runtime.

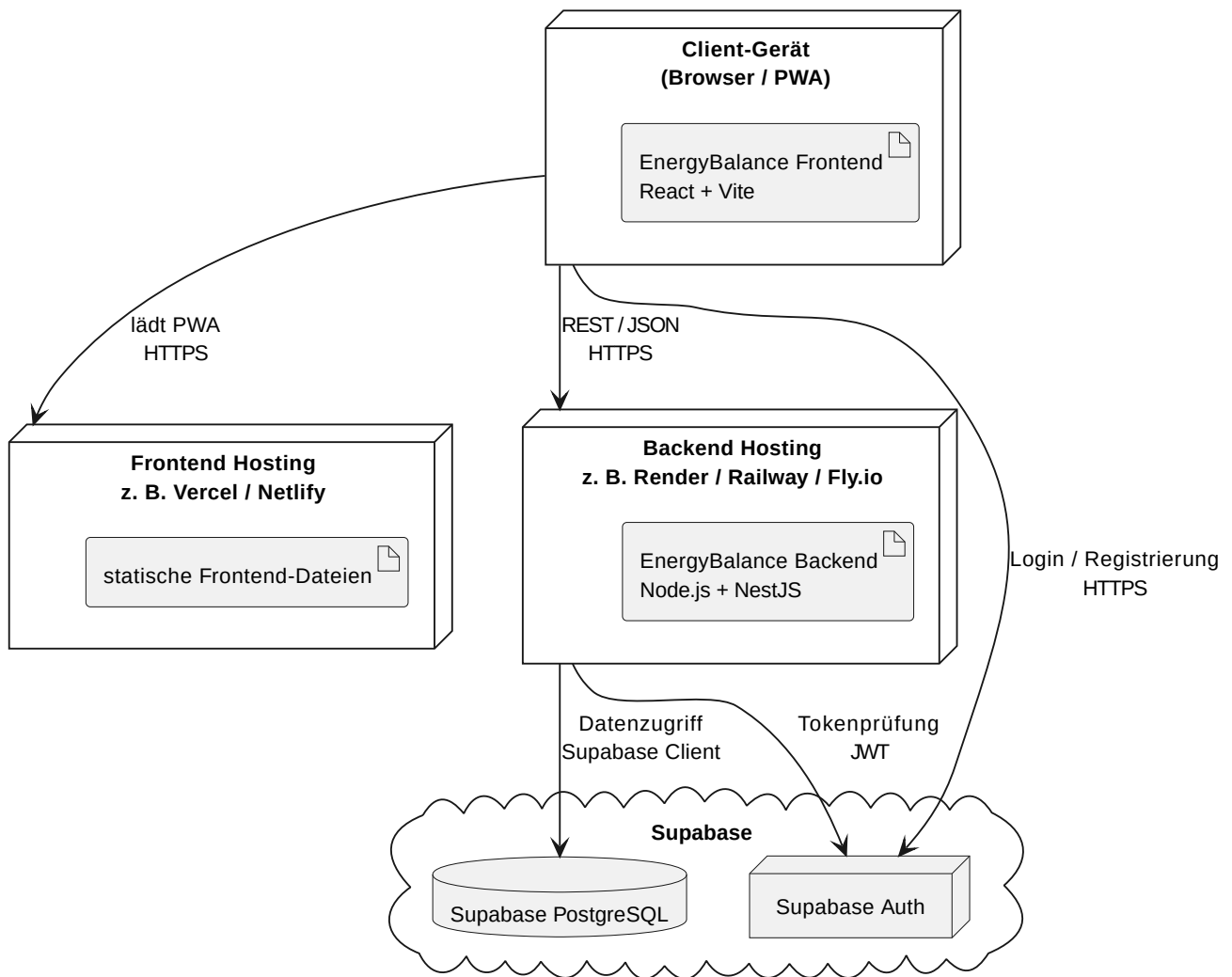
Container Diagram - EnergyBalance



Legend

person
system
container
external person
external system
external container

5.1.3. Verteilungsdiagramm (Deployment Diagramm)



5.2. Technologie-Stack

Dieser Abschnitt listet die eingesetzten Technologien und begründet die Wahl. Detaillierte Architektur-entscheidungen mit Alternativen und Konsequenzen sind in [Section 5.4, "Architekturentscheidungen"](#) dokumentiert.

5.2.1. Übersicht

Bereich	Technologie	Begründung	Entscheidung
Frontend	React + Java-Script + Vite	Entwicklung der Benutzeroberfläche als Progressive Web App (PWA). React ermöglicht eine komponentenbasierte Entwicklung, während Vite eine schnelle Entwicklungsumgebung und kurze Build-Zeiten bietet.	ADR-003

Bereich	Technologie	Begründung	Entscheidung
Backend	Node.js + NestJS	Bereitstellung der REST-API und Umsetzung der Fachlogik. Passt zum JavaScript-Stack und ermöglicht eine klar strukturierte Backend-Architektur mit Controllern, Services und Modulen.	ADR-004
Datenzugriff	Supabase Client	Zugriff auf relationale Daten. Supabase Client ist einfach zu nutzen und passt direkt zu Supabase.	ADR-005
Datenbank	Supabase PostgreSQL	Speicherung strukturierter Daten. PostgreSQL passt zu relationalen Daten wie Nutzerprofilen, Einträgen, Aufgaben, Reflexionen oder Statuswerten.	ADR-006
Authentifizierung	Supabase Auth	Registrierung, Login und Sitzungen. Spart Eigenbau einer sicherheitskritischen Authentifizierung.	ADR-007
API-Kommunikation	REST API mit JSON	Kommunikation zwischen Frontend und Backend. Standardisiert, aus dem Studium bekannt, gut testbar und klar dokumentierbar.	ADR-008
Versionsverwaltung	GitHub	Teamarbeit, Issues und Pull Requests. Zentral für gemeinsame Entwicklung, Nachvollziehbarkeit und Review-Prozess.	ADR-009
CI/CD	GitHub Actions	Automatisierte Builds und Tests können zukünftig über GitHub Actions ausgeführt werden.	ADR-010
Frontend-Testing	Jest oder Vitest React Testing Library	Komponenten- und UI-Tests. Prüft React-Komponenten und Benutzerinteraktionen automatisiert.	ADR-010
Backend-Testing	Jest oder Vitest + Supertest	Backend- und API-Tests. Supertest eignet sich zur automatisierten Überprüfung von REST-Endpunkten des NestJS-Backends.	ADR-010

Bereich	Technologie	Begründung	Entscheidung
Frontend-Hosting	Vercel	Deployment des React/Vite-Frontends. Lässt sich einfach mit GitHub integrieren.	ADR-011
Backend-Hosting	Render, Railway oder Fly.io	Deployment des Node.js/NestJS-Backends. Diese Plattformen sind für Webservices geeignet und einfacher als ein eigener Server.	ADR-011

5.3. Architekturelevante Anforderungen

Dieser Abschnitt dokumentiert, welche Qualitätsziele und Einschränkungen aus dem Produktkonzept direkte Auswirkungen auf Architekturentscheidungen haben.



Qualitätsziele und Einschränkungen werden hier **nicht** neu definiert — sie sind im [Produktkonzept](#) als einziger Quelle festgelegt. Dieser Abschnitt zeigt ausschließlich die technischen Konsequenzen, die sich aus diesen Anforderungen ergeben, und verweist auf die zugehörigen Entscheidungen.

Anforderung (aus Produktkonzept)	Technische Konsequenz	Entscheidung
Benutzbarkeit: Nutzer sollen ihr Energie- und Stresslevel mit maximal 3 Klicks und in unter 15 Sekunden erfassen können.	Das Frontend muss einfach, schnell und mobil nutzbar sein. Eingabeprozesse müssen kurz gehalten und als klar verständliche React-Komponenten umgesetzt werden.	ADR-003
Performance: Das System soll schnell auf Eingaben und Abfragen reagieren; Ladezeiten unter 2 Sekunden bei normalen Nutzerinteraktionen.	Frontend und Backend müssen schlank bleiben. Die REST-API darf keine unnötig großen Datenmengen liefern. Frontend-Build und Hosting müssen schnelle Auslieferung unterstützen.	ADR-003, ADR-008, ADR-011
Verfügbarkeit: Das System soll zuverlässig nutzbar sein.	Frontend und Backend müssen stabil bereitgestellt werden. Hosting-Plattformen müssen zuverlässig genug für das Projektinkrement sein. Deployment darf nicht nur lokal funktionieren.	ADR-006, ADR-007

Anforderung (aus Produktkonzept)	Technische Konsequenz	Entscheidung
Datenschutz: Nutzer müssen Kontrolle über persönliche Daten haben. Die App erfasst nur notwendige Daten. Nutzer können eigene Daten einsehen, exportieren und dauerhaft löschen.	Das Datenmodell muss personenbezogene Daten sauber einem Nutzer zuordnen. Löschung, Datenexport und minimale Datenspeicherung müssen technisch möglich sein.	ADR-003
Zuverlässigkeit der Daten: Eingetragene Werte dürfen nicht verloren gehen, auch bei schlechter Internetverbindung oder App-Absturz.	Es muss geprüft werden, ob lokale Zwischenspeicherung oder Offline-Erfassung nötig ist. Die Synchronisation mit Backend und Datenbank muss robust geplant werden.	ADR-003, ADR-006, ADR-008
Akzeptanz und Motivation: Studierende sollen die App regelmäßig und freiwillig nutzen. Gamification und Reminder sollen Kontinuität fördern.	Das Frontend muss motivierende Elemente und einfache wiederkehrende Nutzung unterstützen. Backend und Datenbank müssen regelmäßige Einträge, Fortschritt oder Statuswerte speichern können.	ADR-002
Projektlaufzeit: Das Produkt wird in zwei Semestern umgesetzt, gegliedert in Sprints. Jeder Sprint endet mit einem Review.	Die Architektur muss inkrementell umsetzbar sein. Zuerst sollte ein Walking Skeleton entstehen: React-Frontend → NestJS-Backend → Supabase PostgreSQL	ADR-002
DSGVO-Konformität: Personenbezogene Daten dürfen nur gemäß DSGVO verarbeitet und gespeichert werden.	Es dürfen nur notwendige personenbezogene Daten gespeichert werden. Zugriff, Löschung, Export und Rollen/Rechte müssen früh berücksichtigt werden.	ADR-006, ADR-007
Hochschulkontext: Das System ist auf Studierende und deren Alltag ausgerichtet.	Die Anwendung muss niederschwellig, webbasiert und ohne Spezialhardware nutzbar sein. Eine PWA/responsive Web-App ist passender als eine Native-Speziallösung.	ADR-002, ADR-003
GitHub und Scrum-Prozess: Versionskontrolle erfolgt ausschließlich über GitHub. Aufgaben werden als Issues im GitHub Project Board verwaltet.	GitHub muss als zentrale Plattform für Code, Dokumentation, Issues, Pull Requests und Review-Prozess verwendet werden.	ADR-009

Anforderung (aus Produktkonzept)	Technische Konsequenz	Entscheidung
Verantwortungsvoller KI-Einsatz: KI-Werkzeuge dürfen unterstützen, müssen aber transparent gemacht und kritisch geprüft werden.	KI-generierte Inhalte dürfen nicht ungeprüft übernommen werden. Die Nutzung muss im Repository bzw. in der Dokumentation nachvollziehbar gemacht werden.	kein ADR erforderlich
Technische Infrastruktur: Einsatz gängiger Webtechnologien, bestehend aus Frontend und Backend, ohne Spezialhardware.	Eine klassische Webarchitektur mit React/Vite-Frontend, Node.js/NestJS-Backend und Supabase-Datenbank ist ausreichend. Spezialhardware oder native-App-Technologien sind nicht notwendig.	ADR002, ADR-003, ADR-004, ADR-006
Programmiersprache: Verwendung bekannter, bzw. geeigneter Technologien aus dem Teamkontext.	JavaScript kann im Frontend und Backend genutzt werden. Dadurch wird der Stack einheitlicher und die Spring-Komplexität entfällt.	ADR-003, ADR-004
Offline-Grundfähigkeit: Einträge müssen auch ohne aktive Internetverbindung erfassbar sein: Sobald Internet vorhanden ist, sollen Daten automatisch hochgeladen werden.	Offline-Erfassung, lokale Zwischenspeicherung und spätere Synchronisation müssen technisch geprüft werden. Das ist eine eigene Architekturfrage und sollte mindestens als Risiko dokumentiert werden.	ADR-003, ADR-008, ggf. neue ADR für Offline-Synchronisation.

5.4. Architekturentscheidungen

Architekturentscheidungen (Architecture Decision Records, ADRs) dokumentieren wichtige technische Entscheidungen: Was wurde entschieden, warum, welche Alternativen wurden verworfen, und welche Konsequenzen hat die Entscheidung.

Jede Entscheidung, die schwer rückgängig zu machen ist oder das gesamte System betrifft, sollte hier festgehalten werden. Kleine, lokale Implementierungsdetails gehören nicht hierher.



ADRs werden nicht gelöscht oder überschrieben — sie werden als „ersetzt durch ADR-XXX“ markiert. So bleibt die Entscheidungshistorie nachvollziehbar.

5.4.1. ADR-002 Wahl der grundlegenden Systemarchitektur

Status	Akzeptiert
Datum	28.05.2026

Kontext	Wir entwickeln eine Webanwendung, die eine klare Trennung zwischen Benutzeroberfläche, Fachlogik und Datenhaltung benötigt. Das Team arbeitet mit JavaScript im Frontend und Backend. Die Anwendung soll wartbar, testbar und innerhalb des Projektrahmens realistisch umsetzbar sein.
Entscheidung	Wir verwenden React/Vite im Frontend, Node.js/NestJS im Backend und Supabase PostgreSQL für die Datenhaltung. Frontend und Backend kommunizieren über eine REST-API mit JSON.
Alternativen	Supabase only ohne eigenes Backend; Spring Boot als Backend; klassische serverseitige Webanwendung ohne getrenntes Frontend.
Konsequenzen	Die Architektur ist klar getrennt und unterstützt eine wartbare Entwicklung. Durch JavaScript im Frontend und Backend entsteht ein einheitlicher Technologie-Stack. Frontend, Backend und Datenbank müssen jedoch getrennt entwickelt, integriert und getestet werden.

5.4.2. ADR-003: Verwendung von React, JavaScript und Vite im Frontend

Status	Akzeptiert
Datum	28.05.2026
Kontext	Die Anwendung benötigt eine moderne und interaktive Benutzeroberfläche. Das Team hat erste Erfahrungen mit JavaScript und React. Die Anwendung soll für mobile Geräte geeignet sein und perspektivisch als PWA funktionieren.
Entscheidung	Wir verwenden React mit JavaScript und Vite für das Frontend.
Alternativen	Vue.js/Nuxt.js; React Native; Thymeleaf mit Spring Boot; serverseitiges Rendering mit Templates.
Konsequenzen	React ermöglicht komponentenbasierte Entwicklung der Benutzeroberfläche. Vite vereinfacht Entwicklung und Build-Prozess. Das Risiko ist, dass Fachlogik versehentlich in UI-Komponenten landen. Deshalb müssen Komponenten, Services und API-Zugriffe getrennt werden.

5.4.3. ADR-004: Verwendung von Node.js und NestJS als Backend

Status	Akzeptiert
Datum	28.05.2026
Kontext	Die Anwendung benötigt eine zentrale Backend-Schicht für Fachlogik, Validierung und REST-Schnittstellen. Das Team verwendet bereits JavaScript im Frontend. Ein JavaScript-basiertes Backend reduziert den Technologiewechsel zwischen Frontend und Backend und erleichtert die Zusammenarbeit.

Entscheidung	Wir verwenden Node.js mit NestJS als Backend. Die Fachlogik wird in Services umgesetzt. Controller stellen REST-Endpunkte für das Frontend bereit.
Alternativen	Spring Boot als Backend; Supabase ohne eigenes Backend; Node.js mit Express.
Konsequenzen	NestJS bietet eine klare Projektstruktur mit Controllern, Services und Modulen. Dadurch wird die Trennung von Verantwortlichkeiten unterstützt. Der Nachteil ist ein höherer Einstieg als bei Express, insbesondere durch die stärkeren Framework-Konventionen.

5.4.4. ADR-005: Verwendung des Supabase Clients für den Datenzugriff

Status	Akzeptiert
Datum	28.05.2026
Kontext	Das Backend muss auf die Supabase PostgreSQL gespeicherten Daten zugreifen. Dafür wird eine einfache und projektgeeignete Zugriffsschicht benötigt.
Entscheidung	Wir verwenden den Supabase Client für den Datenzugriff aus dem Node.js/NestJS-Backend.
Alternativen	Prisma; node-postgres; direkter Datenbankzugriff aus dem Frontend.
Konsequenzen	Der Supabase Client passt direkt zur gewählten Datenbankplattform und reduziert den Einrichtungsaufwand. Der Nachteil ist eine stärkere Bindung an Supabase-spezifische Schnittstellen. Bei wachsender Komplexität kann später ein strukturierter Zugriff über Prisma geprüft werden.

5.4.5. ADR-006: Verwendung von Supabase PostgreSQL als Datenbank

Status	Akzeptiert
Datum	28.05.2026
Kontext	Die Anwendung benötigt eine relationale Datenbank für strukturierte Daten, zum Beispiel für Nutzerprofile, Einträge, Aufgaben, Statuswerte oder Reflexionen. Relationale Daten passen gut zu den erwarteten Datenstrukturen.
Entscheidung	Wir verwenden Supabase PostgreSQL als Datenbank.
Alternativen	Lokale PostgreSQL-Datenbank; Firebase; SQLite; MySQL
Konsequenzen	PostgreSQL passt gut zu relationalen Daten. Supabase reduziert den Aufwand für Einrichtung und Betrieb der Datenbank. Das Risiko ist die Abhängigkeit von Supabase als externen Dienst. Datenmodell, Rechte und Backups müssen bewusst geplant werden.

5.4.6. ADR-007: Verwendung von Supabase Auth für Authentifizierung

Status	Akzeptiert
Datum	28.05.2026
Kontext	Die Anwendung benötigt Registrierung, Login und Sitzungsverwaltung. Authentifizierung ist sicherheitskritisch und sollte nicht unnötig selbst implementiert werden.
Entscheidung	Wir verwenden Supabase Auth für Registrierung, Login und Sitzungsverwaltung.
Alternativen	Eigene Authentifizierung im Node.js/NestJS-Backend; Auth.js; keine Authentifizierung.
Konsequenzen	Supabase Auth spart Entwicklungszeit und reduziert Sicherheitsrisiken durch Eigenbau. Das Team muss aber klären, wie das Frontend Authentifizierungsinformationen an das Backend weitergibt und wie das Backend diese prüft. Rollen und Rechte müssen sauber definiert werden.

5.4.7. ADR-008: Kommunikation über REST API mit JSON

Status	Akzeptiert
Datum	28.05.2026
Kontext	Frontend und Backend werden getrennt entwickelt. Deshalb wird eine klar definierte Schnittstelle benötigt, über die Daten zwischen React/Vite und dem NestJS-Backend ausgetauscht werden.
Entscheidung	Die Kommunikation zwischen Frontend und Backend erfolgt über eine REST API mit JSON.
Alternativen	GraphQL; direkte Kommunikation des Frontends mit der Datenbank; serverseitiges Rendering ohne API.
Konsequenzen	REST mit JSON ist verständlich, testbar und gut dokumentierbar. Die API-Endpunkte müssen aber früh abgestimmt werden, damit Frontend und Backend unabhängig arbeiten können. Änderungen an Schnittstellen müssen im Team kommuniziert werden.

5.4.8. ADR-009: Verwendung von GitHub für Versionsverwaltung und Zusammenarbeit

Status	Akzeptiert
Datum	28.05.2026
Kontext	Das Projekt wird im Team entwickelt. Änderungen am Code, an Dokumentation und Architekturentscheidungen müssen nachvollziehbar sein. Außerdem braucht das Team eine gemeinsame Plattform für Pull Requests, Issues und Review-Prozesse.

Entscheidung	Wir verwenden GitHub als zentrale Plattform für Versionsverwaltung, Zusammenarbeit, Issues und Pull-Requests.
Alternativen	GitLab, lokale Git-Repositories ohne zentrale Plattform, manuelle Dateiverwaltung.
Konsequenzen	Änderungen sind nachvollziehbar und können im Team überprüft werden. Pull Requests unterstützen Code Reviews. Der Nachteil ist, dass das Team Git-Workflows wie Branches, Commits, Pull Requests und Merge Konflikte sicher beherrschen muss.

5.4.9. ADR-010: Automatisierte Qualitätssicherung mit GitHub Actions, Jest/Vitest und Supertest

Status	Vorgeschlagen
Datum	28.05.2026
Kontext	Frontend und Backend sollen nicht nur manuell geprüft werden. Fehler sollen früh erkannt werden, bevor Änderungen in den Hauptstand übernommen werden. Dafür werden automatisierte Tests und ein automatisierter Prüfprozess benötigt.
Entscheidung	Wir verwenden GitHub Actions für automatisierte Builds und Tests. Frontend-Komponenten werden mit Jest oder Vitest und React Testing Library getestet. Backend- und API-Funktionen werden mit Jest oder Vitest und Supertest getestet.
Alternativen	Nur manuelle Tests; lokale Tests ohne CI/CD; andere CI/CD-Plattformen wie GitLab CI.
Konsequenzen	Änderungen können automatisch geprüft werden. Die Codequalität und Stabilität der Anwendung steigen. Der Nachteil ist zusätzlicher Einrichtungsaufwand für Testskripte, Workflows und Testdaten. Außerdem müssen fehlgeschlagene Tests im Team konsequent behoben werden.

5.4.10. ADR-011: Deployment- und Hostingstrategie für Frontend und Backend

Status	Vorgeschlagen
Datum	28.05.2026
Kontext	Die Anwendung soll nicht nur lokal laufen, sondern auch als nutzbares Produktinkrement bereitgestellt werden. Frontend und Backend werden getrennt entwickelt und benötigen daher passende Hosting-Optionen.
Entscheidung	Das Frontend wird über Vercel oder Netlify bereitgestellt. Das Node.js/NestJS-Backend wird über Render, Railway oder Fly.io bereitgestellt.
Alternativen	Lokale Ausführung ohne Deployment, eigener Server, Hochschulserver, vollständiges Deployment über eine einzige Plattform.

Konsequenzen	Frontend und Backend können unabhängig bereitgestellt werden. Vercel und Netlify sind gut für React/Vite-Projekte geeignet. Render, Railway und Fly.io eignen sich für Webservices und Docker-basierte Deployments. Der Nachteil ist zusätzlicher Konfigurationsaufwand für Umgebungsvariablen, Authentifizierung, Datenbankzugriff und Deployment-Pipeline.
---------------------	--



Weitere ADRs nach demselben Schema hinzufügen. Die Nummerierung ist fortlaufend und wird nie wiederverwendet.

5.5. Risiken & technische Schulden

Dieser Abschnitt dokumentiert bekannte technische Risiken und bewusst eingegangene technische Kompromisse (Technical Debt). Transparenz hier verhindert, dass diese Punkte in Vergessenheit geraten.

5.5.1. Technische Risiken

Risiko	Auswirkung	Wahrscheinlichkeit	Maßnahme
Integration zwischen React/Vite-Frontend und Node.js/NestJS-Backend funktioniert nicht reibungslos.	Hoch	Mittel	Früh einen technischen Durchstich erstellen und die Kommunikation zwischen Frontend und Backend testen.
Authentifizierung mit Supabase Auth und NestJS-Backend ist unklar.	Hoch	Mittel	Früh klären, wie Login-Token vom Frontend an das Backend übergeben und dort geprüft werden. Auth-Flow dokumentieren und testen.
Zugriff des NestJS-Backends auf Supabase PostgreSQL verursacht Konfigurationsprobleme.	Hoch	Mittel	Datenbankverbindung früh testen. Verbindung, Supabase-URL, API-Keys, Umgebungsvariablen und Zugriffsdaten dokumentieren.
Fachlogik landet versehentlich im Frontend statt im Backend	Mittel	Niedrig	Klare Trennung zwischen React-Komponenten, NestJS-Services und Datenzugriff einhalten.

Risiko	Auswirkung	Wahrscheinlichkeit	Maßnahme
REST API wird nicht früh genug abgestimmt	Mittel	Mittel	API-Endpunkte früh definieren, z.B. Pfade, Request-Body, Response-Body und Fehlerfälle. Optional einfache API-Dokumentation erstellen.
Team unterschätzt Aufwand durch getrenntes Frontend, Backend und Datenbank	Hoch	Hoch	MVP klein halten. Erst Walking Skeleton bauen, danach Funktionen erweitern. Keine unnötigen Zusatzfeatures in Sprint 2.
Deployment von Frontend und Backend verursacht Probleme	Mittel	Mittel	Deployment erst als vorgeschlagen behandeln. Vor finaler Entscheidung einen Deployment-Test mit Vercel/Netlify und Render/Railway/Fly.io durchführen.
CORS-Probleme zwischen Frontend und Backend	Mittel	Mittel	CORS-Konfiguration im NestJS-Backend früh testen. Lokale und spätere Deployment-URLs berücksichtigen.
GitHub Actions / automatisierte Tests funktionieren nicht direkt	Niedrig	Mittel	CI/CD zunächst minimal aufsetzen: Projekt bauen und einfache Tests ausführen. Komplexe Workflows später ergänzen.
Backend-Testing mit Jest/Vitest und Supertest wird nicht konsequent umgesetzt	Mittel	Mittel	Mindestens zentrale API-Endpunkte automatisiert testen. Tests nicht erst am Ende schreiben.
Supabase-Abhängigkeit führt zu Vendor-Lock-in	Mittel	Mittel	Supabase bewusst nur für Datenbank/Auth nutzen. Fachlogik im NestJS-Backend halten, damit ein Wechsel später realistischer bleibt.

Risiko	Auswirkung	Wahrscheinlichkeit	Maßnahme
Sicherheitsregeln/ Rollen sind unklar	Hoch	Mittel	Rollenmodell früh definieren. Prüfen, welche Daten welcher Nutzer sehen oder ändern darf. Zugriff nicht nur im Frontend absichern.
Hochschulserver verursacht mehr Aufwand als Nutzen	Mittel	Mittel	Hochschulserver nur nutzen, wenn Zugang, SSL, Ports, Deployment und Support klar geregelt sind. Sonst Render/Railway/Fly.io bevorzugen.
Offline-Erfassung und spätere Synchronisation sind technisch aufwendiger als erwartet	Hoch	Mittel	Offline-Fähigkeit zunächst als eigenes Risiko behandeln. Früh entscheiden, ob sie Teil des MVP ist oder später umgesetzt wird.

5.5.2. Technische Schulden

Beschreibung	Begründung für Kompromiss	Backlog-Eintrag
Automatisierte Tests sind noch nicht vollständig umgesetzt.	Fokus lag zunächst auf der Fertigstellung des ersten Produktinkrements. Die Testabdeckung wird in späteren Sprints erweitert.	-
Fehlerbehandlung deckt bisher hauptsächlich Standardfälle ab.	Erweiterung erfolgt in späteren Sprints.	-
Deployment-Pipeline ist noch nicht vollständig automatisiert.	Zunächst lokale Entwicklung und Integration priorisiert.	-

6. Softwaredesign: F2 - EnergyBalance

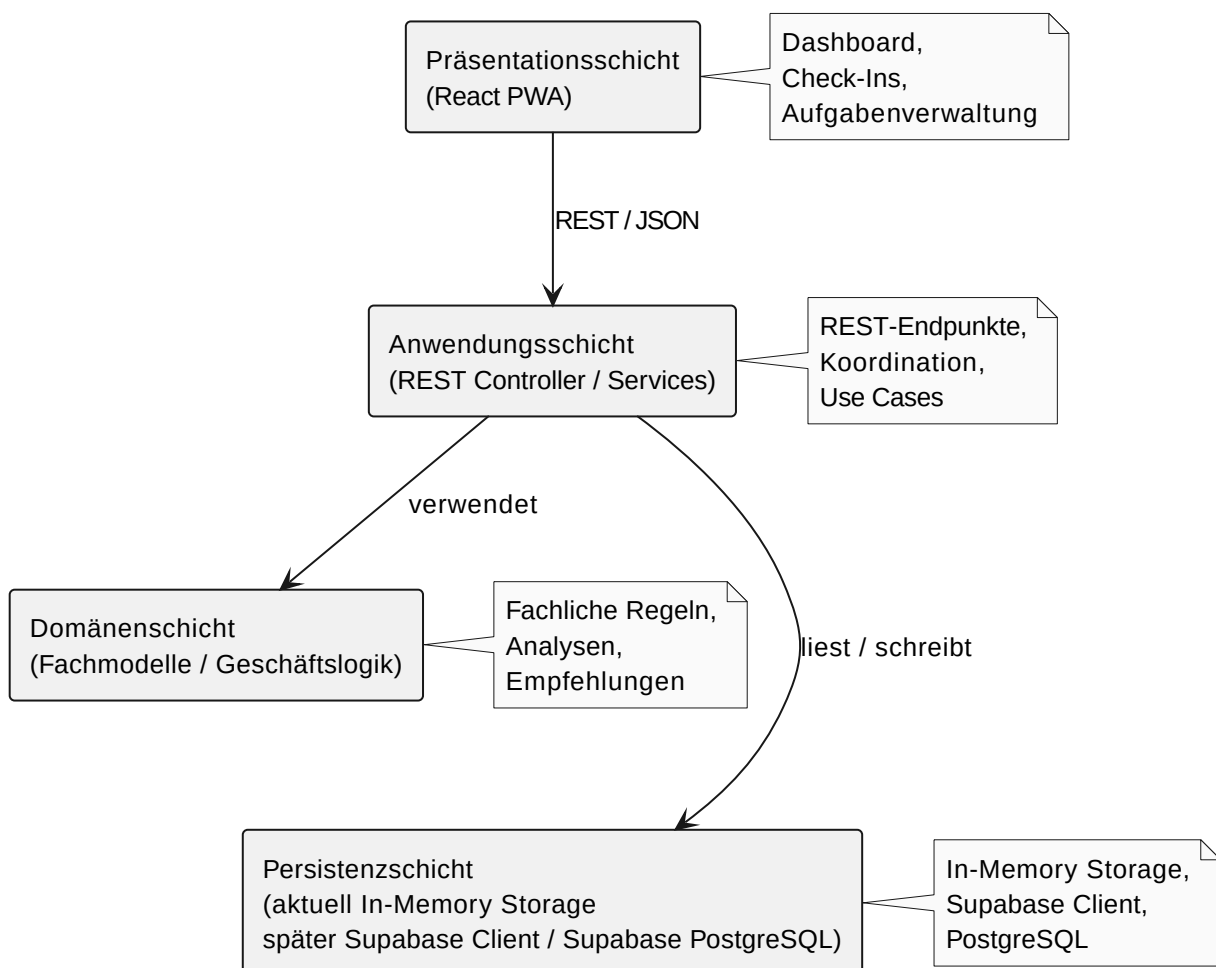
Dieses Dokument beschreibt die interne Struktur des Systems, wie der Code organisiert ist, aus welchen Komponenten das System besteht, wie diese zusammenarbeiten, und welche Designmuster eingesetzt werden. Der Fokus liegt auf dem **Wie** der Implementierung.

Das Systemkontext-Diagramm und die Architekturentscheidungen (ADRs) sind im [Architekturdokument](#) ([architecture.adoc](#)) beschrieben. Dieses Dokument vertieft die interne Sicht. Das Datenmodell hier beschreibt die technische Struktur der Persistenz. Die fachliche Bedeutung der Entitäten ist im [Domänenmodell](#) ([domain_glossary.adoc](#)) definiert.

6.1. Systemübersicht & Schichtenarchitektur

Dieser Abschnitt beschreibt, in welche Schichten oder Bereiche das System aufgeteilt ist und welche Verantwortung jede Schicht trägt. Eine klare Schichtentrennung verhindert, dass Geschäftslogik, Datenzugriff und Präsentation unkontrolliert vermischt werden.

6.1.1. Schichtendiagramm



6.1.2. Beschreibung der Schichten

Schicht	Verantwortung	Darf abhängen von
Präsentations-schicht	Darstellung der Benutzeroberfläche; Erfassung von Check-Ins, Aufgaben und Reflexionen; Anzeige von Analysen, Warnungen und Empfehlungen; Kommunikation mit dem Backend über REST/JSON	Anwendungsschicht
Anwendungs-schicht	Verarbeitung von HTTP-Anfragen; Bereitstellung von REST-Endpunkten; Implementierung von Use Cases; Koordination von Geschäftslogik, Authentifizierung und Datenzugriff	Domänenschicht, Persistenzschicht
Domänenschicht	Fachliche Modelle wie Nutzer, Check-In, Aufgabe und Reflexion; fachliche Regeln zur Auswertung von Stress-, Energie- und Aktivitätsdaten sowie zur Ableitung von Analysen, Warnungen und Empfehlungen	Keine äußeren Schichten
Persistenzschicht	Speicherung und Abruf von Anwendungsdaten; Persistenz über Supabase Client und Supabase PostgreSQL	Keine



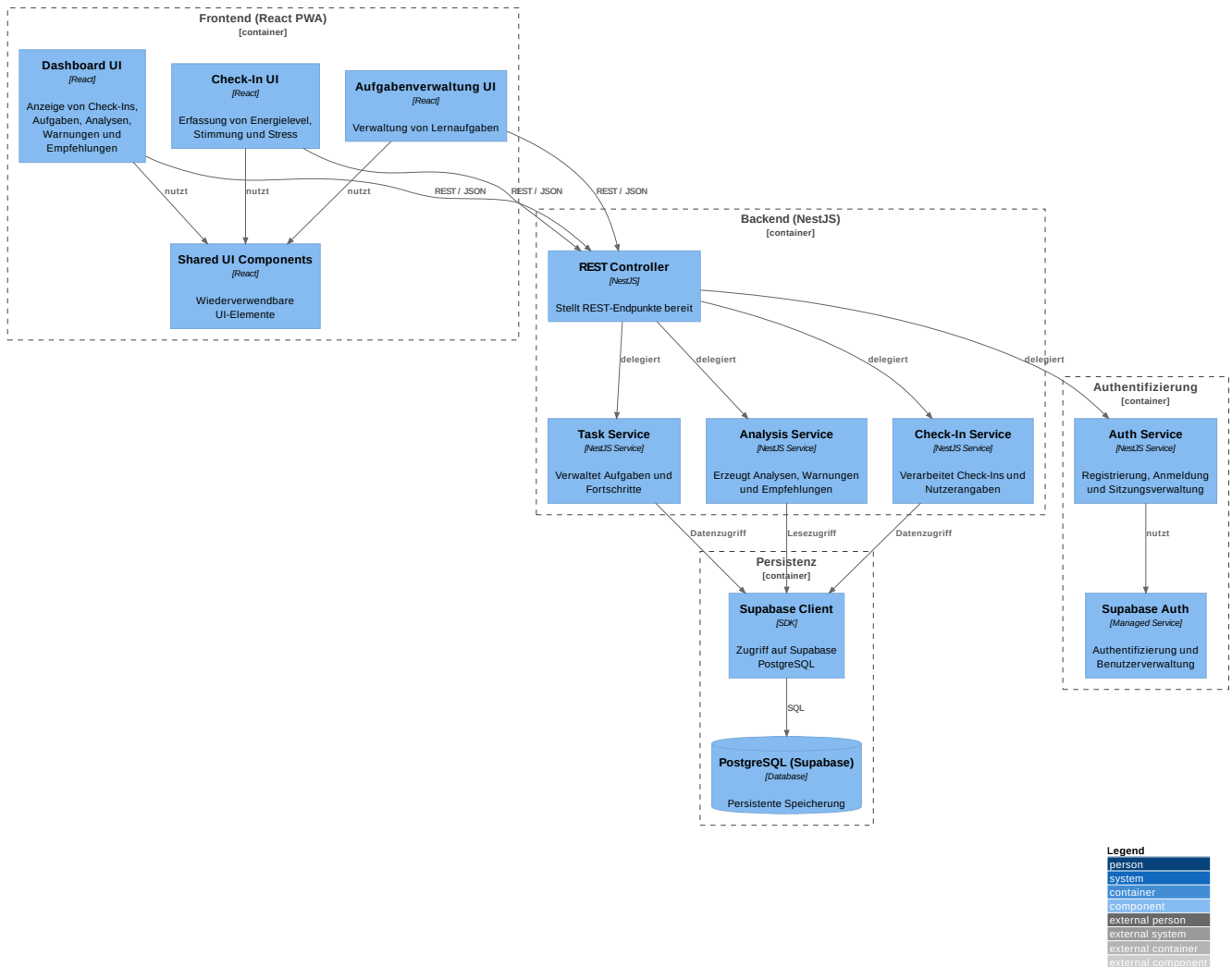
Die Domänenschicht darf **nicht** von Persistenz- oder Präsentationsschicht abhängen. Diese Regel ist der häufigste Architekturverstoss in Studentenprojekten und sollte in Code Reviews explizit geprüft werden.

6.2. Komponentenübersicht

Dieser Abschnitt beschreibt die wichtigsten Komponenten des Systems — Module, Pakete oder Services — und ihre Verantwortlichkeiten. Eine Komponente ist eine zusammengehörige Einheit von Code mit einer klar definierten Aufgabe.

6.2.1. Komponentendiagramm

Component Diagram - EnergyBalance



6.2.2. Komponentenbeschreibung

Komponente	Verantwortung	Schnittstellen
REST Controller	Entgegennahme von HTTP-Anfragen; Routing; Validierung von Eingabedaten; Rückgabe von JSON-Antworten	Frontend; Services
Auth Service	Registrierung, Anmeldung und Sitzungsverwaltung über Supabase Auth; Verwaltung authentifizierter Benutzerzugriffe	REST Controller; Supabase Auth
Check-In Service	Verwaltung von Check-Ins; Verarbeitung von Energielevel, Stresswerten, Stimmung und Reflexionen	REST Controller; Supabase Client
Task Service	Erstellung, Bearbeitung und Abruf von Aufgaben; Verwaltung des Bearbeitungsstatus	REST Controller; Supabase Client

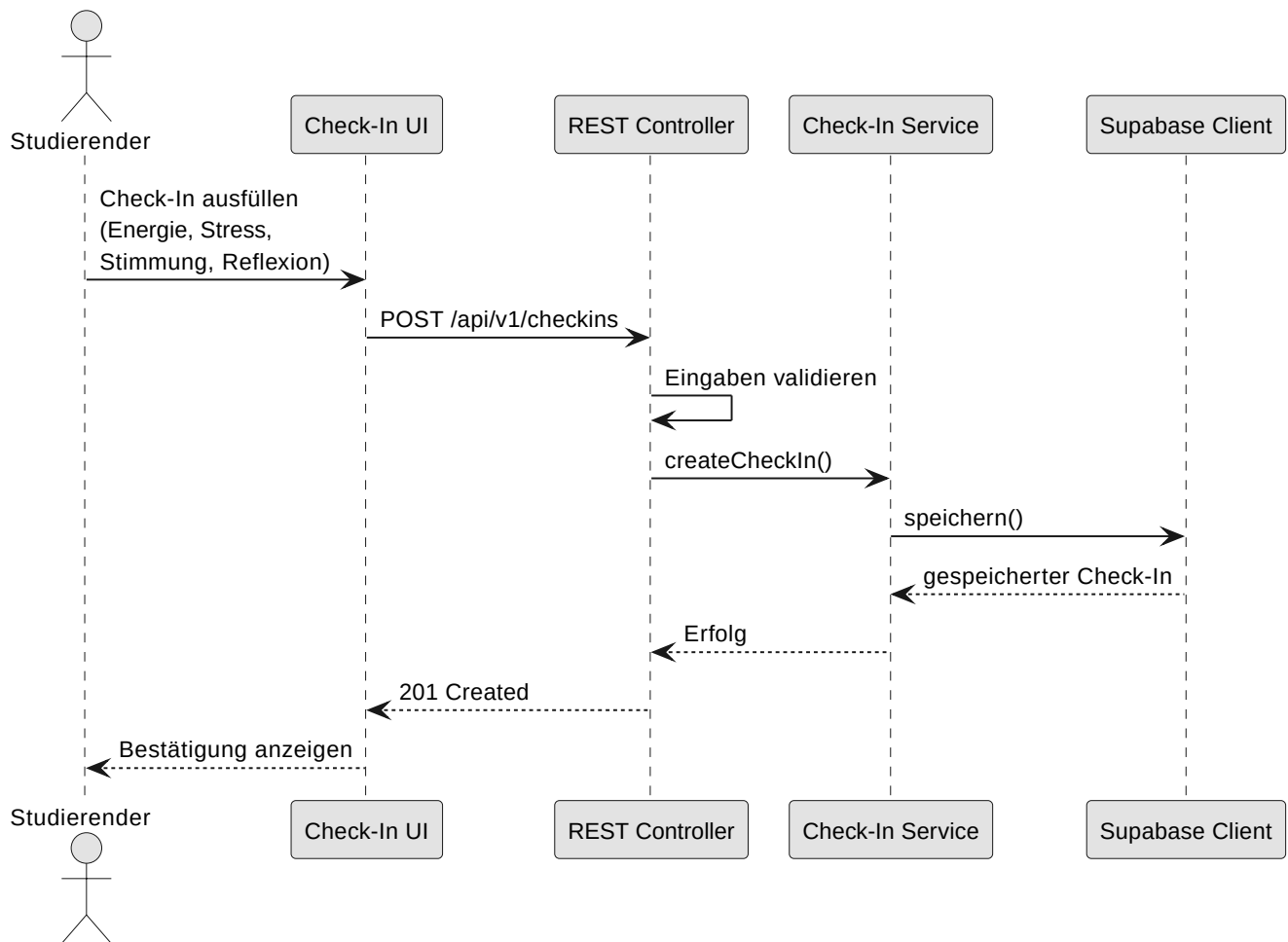
Komponente	Verantwortung	Schnittstellen
Analysis Service	Berechnung von Trends; Analyse von Stress-, Energie- und Aktivitätsdaten; Generierung von Warnungen und Empfehlungen	REST Controller; Supabase Client
Shared UI Components	Wiederverwendbare Benutzeroberflächen-Komponenten wie Buttons, Formulare, Dialoge und Layouts	Dashboard UI; Check-In UI; Aufgabenverwaltung UI
Supabase Client	Zugriff auf die PostgreSQL-Datenbank über das Supabase SDK; Ausführung von Datenbankoperationen	Services; PostgreSQL
Supabase Auth	Benutzerverwaltung, Authentifizierung und Sitzungsverwaltung	Auth Service
PostgreSQL (Supabase)	Persistente Speicherung aller Anwendungsdaten	Supabase Client

6.3. Wichtige Interaktionsflüsse

Sequenzdiagramme zeigen, wie Komponenten bei typischen Abläufen zusammenarbeiten. Dokumentiert werden hier nur die wichtigsten und komplexesten Flüsse — nicht jede einfache CRUD-Operation.

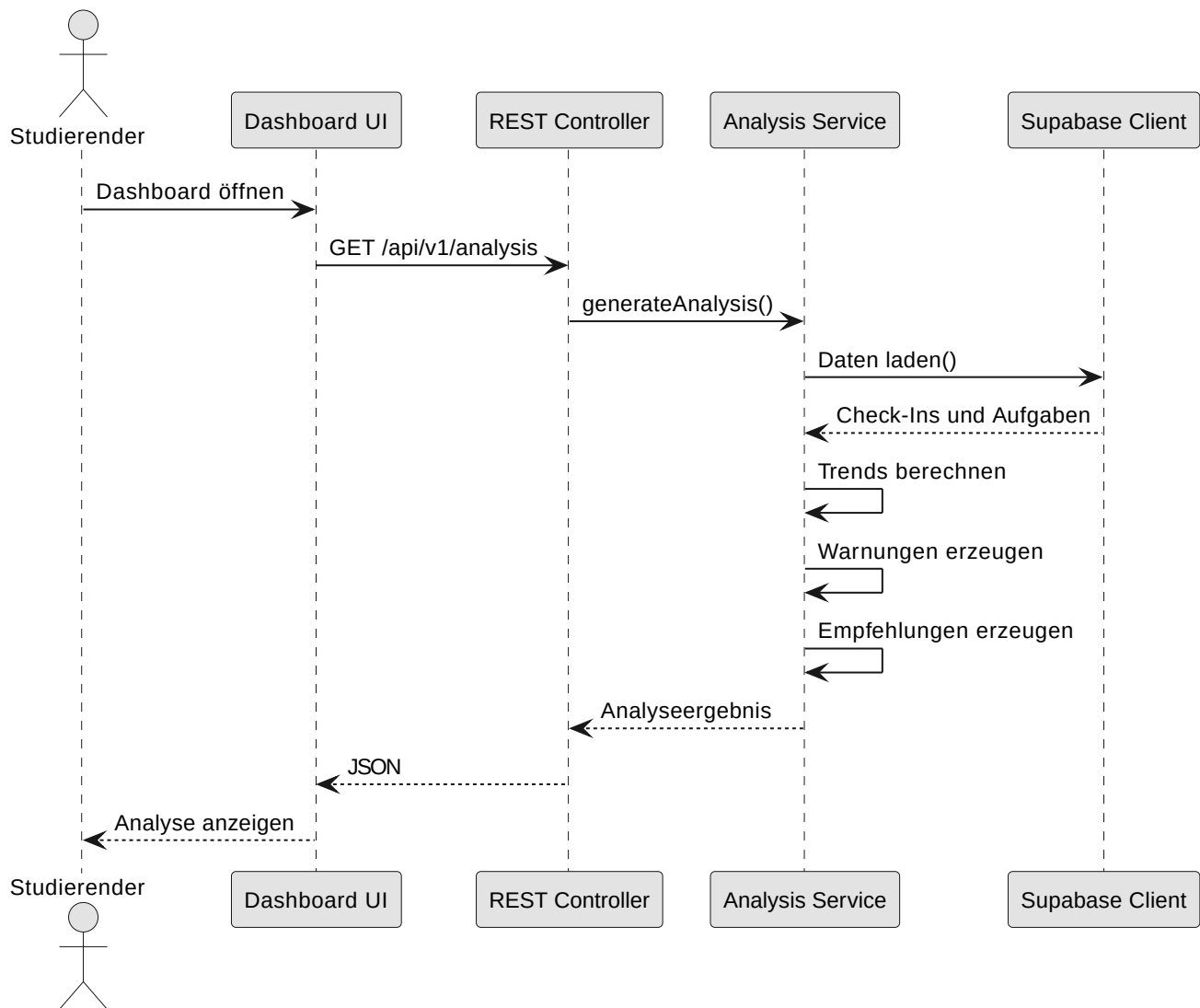
6.3.1. Check-In erfassen

Dieser Ablauf zeigt die Erfassung eines täglichen Check-Ins durch einen Studierenden. Die eingegebenen Werte werden validiert und gespeichert. Die gespeicherten Daten bilden die Grundlage für spätere Analysen, Warnungen und Empfehlungen.



6.3.2. Analyse anzeigen

Dieser Ablauf zeigt die Erstellung einer Analyse auf Basis gespeicherter Check-Ins und Aufgaben. Dabei werden Trends erkannt und daraus Warnungen sowie Empfehlungen abgeleitet.



Flussnamen im Diagramm sollten identisch mit den Methodennamen im Code sein. Das macht es einfacher, vom Diagramm zum Code zu navigieren und umgekehrt.

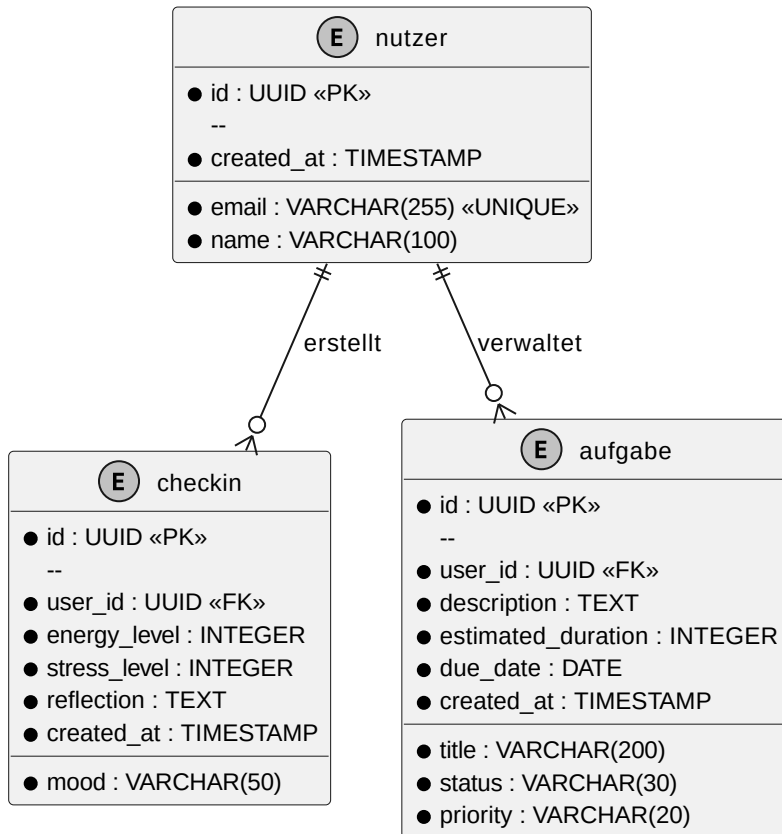
6.4. Datenmodell

Dieser Abschnitt beschreibt die technische Datenbankstruktur: Tabellen, Spalten, Datentypen und Beziehungen. Er beantwortet die Frage: Wie werden die Daten gespeichert?



Die **fachliche** Bedeutung der Entitäten — was sie repräsentieren und wie sie sich zueinander verhalten — ist im Domänenmodell ([domain_glossary.adoc](#)) beschrieben. Dieses Dokument zeigt ausschließlich die technische Persistenzstruktur.

6.4.1. Datenbankmodell



6.4.2. Tabellenbeschreibung

Tabelle	Spalte	Beschreibung & Besonderheiten
nutzer	id	UUID als Primärschlüssel; wird von Supabase Auth generiert
	email	Eindeutige E-Mail-Adresse des Nutzers
	name	Anzeigenname des Nutzers
	created_at	Zeitpunkt der Benutzererstellung
checkin	id	UUID als Primärschlüssel
	user_id	Fremdschlüssel auf den Nutzer
	energy_level	Energielevel des Nutzers, typischerweise auf einer Skala von 1 bis 10
	stress_level	Wahrgenommener Stresslevel, typischerweise auf einer Skala von 1 bis 10
	mood	Aktuelle Stimmung des Nutzers
	reflection	Freitextfeld für persönliche Reflexionen
	created_at	Zeitpunkt des Check-Ins

Tabelle	Spalte	Beschreibung & Besonderheiten
aufgabe	id	UUID als Primärschlüssel
	user_id	Fremdschlüssel auf den Nutzer
	title	Kurzer Titel der Aufgabe
	description	Detaillierte Beschreibung der Aufgabe
	status	Bearbeitungsstatus (z. B. OFFEN, IN_BEARBEITUNG, ERLEDIGT)
	priority	Priorität der Aufgabe (z. B. NIEDRIG, MITTEL, HOCH)
	estimated_duration	Geschätzte Bearbeitungsdauer in Minuten
	due_date	Optionales Fälligkeitsdatum
	created_at	Zeitpunkt der Erstellung

6.4.3. Datenbankmigrationen

Schemaänderungen werden nicht manuell durchgeführt, sondern über versionierte Migrationskripte verwaltet. Dadurch bleibt die Datenbankstruktur nachvollziehbar, reproduzierbar und zwischen Entwicklungs- und Produktivumgebungen konsistent.

Aspekt	Beschreibung
Werkzeug	Supabase CLI Migrations
Speicherort	<code>supabase/migrations/</code>
Namenskonvention	Zeitstempelbasierte Benennung, z. B. <code>20250610120000_initial_schema.sql</code>
Regel	Bereits ausgeführte Migrationen werden nicht verändert. Änderungen am Schema erfolgen ausschließlich über neue Migrationen.
Deployment	Migrationen werden über die Supabase CLI auf die Zielumgebung angewendet.

6.5. Eingesetzte Muster & Konventionen

Dieser Abschnitt dokumentiert übergreifende Designmuster und Konventionen, die im gesamten Codebase gelten. Das verhindert, dass verschiedene Teammitglieder dasselbe Problem auf unterschiedliche Weise lösen.

6.5.1. Designmuster

Muster	Wo eingesetzt	Begründung
Service Layer	Backend (Check-In Service, Task Service, Analysis Service, Auth Service)	Trennung von HTTP-Verarbeitung und Geschäftslogik; Controller bleiben schlank und delegieren fachliche Aufgaben an Services

Muster	Wo eingesetzt	Begründung
Component-Based Architecture	Frontend (React)	Benutzeroberfläche wird aus wiederverwendbaren Komponenten aufgebaut; verbessert Wartbarkeit und Wiederverwendbarkeit
Separation of Concerns	Gesamte Anwendung	Präsentation, Geschäftslogik und Datenzugriff werden in getrennten Schichten umgesetzt; reduziert Kopplung und erleichtert Erweiterungen
RESTful API Design	Kommunikation zwischen Frontend und Backend	Klare Ressourcenorientierung und standardisierte Kommunikation über HTTP und JSON

6.5.2. Fehlerbehandlung

Aspekt	Konvention
Validierungsfehler	Ungültige oder unvollständige Eingaben werden mit HTTP Statuscode 400 Bad Request beantwortet
Nicht gefundene Ressourcen	Nicht vorhandene Check-Ins oder Aufgaben werden mit 404 Not Found beantwortet
Authentifizierungsfehler	Fehlende oder ungültige Anmeldedaten werden mit 401 Unauthorized beantwortet
Serverfehler	Unerwartete Fehler werden mit 500 Internal Server Error beantwortet
Fehlermeldungen	Fehler werden als strukturierte JSON-Antworten zurückgegeben
Logging	Fehler werden serverseitig protokolliert; sensible Nutzerdaten werden nicht in Logdateien gespeichert

6.5.3. API-Konventionen

Aspekt	Konvention
URL-Struktur	Ressourcenorientierte Endpunkte im Plural, z. B. /api/v1/checkins , /api/v1/tasks , /api/v1/analysis
HTTP-Methoden	GET = lesen; POST = erstellen; PUT = vollständiges Aktualisieren; PATCH = teilweises Aktualisieren; DELETE = löschen
Antwortformat	JSON für alle Anfragen und Antworten
Datumsformat	ISO-8601 Format, z. B. 2025-06-15T14:30:00Z
HTTP-Statuscodes	200 für erfolgreiche Anfragen; 201 für erfolgreich erstellte Ressourcen; 400 für fehlerhafte Eingaben; 401 für Authentifizierungsfehler; 404 für nicht gefundene Ressourcen; 500 für unerwartete Serverfehler
Authentifizierung	Authentifizierte Anfragen verwenden die von Supabase Auth bereitgestellten Tokens

Aspekt	Konvention
Versionierung	Versionierung über URL-Präfix (<i>/api/v1/</i>); neue Hauptversionen nur bei inkompatiblen Änderungen

Diagramme in diesem Dokument sollten den aktuellen Stand des Codes widerspiegeln. Bei größeren Refactorings oder neuen Komponenten ist dieses Dokument im selben Pull Request zu aktualisieren.

7. Testkonzept: F2 - EnergyBalance

Dieses Dokument beschreibt, wie die Qualität der Software sichergestellt wird. Es legt fest, welche Arten von Tests eingesetzt werden, welche Werkzeuge verwendet werden, und wie Tests in den Entwicklungsprozess integriert sind.

7.1. Teststrategie

Beschreibe in wenigen Sätzen den grundsätzlichen Ansatz zur Qualitätssicherung im Projekt. Warum wurden diese Testarten gewählt? Welchen Stellenwert hat Testing im Team?

[Hier die übergeordnete Teststrategie beschreiben, z. B.: „Wir setzen auf eine Teststrategie mit einem starken Fokus auf Unit-Tests, ergänzt durch Integrationstests für kritische Schnittstellen und manuelle Tests für die wichtigsten Nutzerinteraktionen.“]

7.2. Testarten

7.2.1. Unit-Tests

Unit-Tests prüfen einzelne Klassen oder Funktionen isoliert voneinander.

Aspekt	Beschreibung
Ziel	Sicherstellung der korrekten Funktionsweise einzelner Komponenten in Isolation
Umfang	Alle Service- und Domänenklassen; Fokus auf Geschäftslogik
Werkzeug	[z. B. JUnit 5, Pytest, Jest]
Mocking (Faken / Simulieren)	[z. B. Mockito, unittest.mock — externe Abhängigkeiten werden gemockt]
Abdeckungsziel	[z. B. mind. 80 % Zeilenabdeckung auf der Service-Schicht]
Ausführung	Lokal bei jedem Build; automatisch in der CI-Pipeline bei jedem Push

7.2.2. Integrationstests

Integrationstests prüfen das Zusammenspiel mehrerer Komponenten, z. B. Service und Datenbank.

Aspekt	Beschreibung
Ziel	Sicherstellung der korrekten Kommunikation zwischen Komponenten und mit externen Systemen
Umfang	[z. B. REST-Endpunkte, Datenbankzugriffe, externe API-Aufrufe]
Werkzeug	[z. B. Spring Boot Test, Testcontainers, Supertest]
Testdaten	[z. B. dedizierte Testdatenbank, In-Memory-Datenbank, Fixtures]

Aspekt	Beschreibung
Ausführung	Automatisch in der CI-Pipeline; nicht zwingend bei jedem lokalen Build

7.2.3. Optional: End-to-End-Tests (E2E)

E2E-Tests simulieren echte Nutzerinteraktionen durch die gesamte Anwendung.

Aspekt	Beschreibung
Ziel	Sicherstellung der wichtigsten Interaktionen aus Endnutzerperspektive
Umfang	[z. B. Login, Kernfunktion A, Kernfunktion B — nur kritische Pfade]
Werkzeug	[z. B. Cypress, Playwright, Selenium]
Ausführung	[z. B. manuell vor jedem Release; oder automatisch auf Staging-Umgebung]

7.2.4. Manuelle Tests

Hier werden alle Tests aufgeführt, die nicht automatisiert werden können.

Aspekt	Beschreibung
Ziel	Überprüfung von Usability, visueller Korrektheit und explorativem Testen
Umfang	[z. B. UI-Design, Responsivität, neue Features vor dem Sprint Review]
Vorgehen	[z. B. Exploratives Testen durch Teammitglied, das das Feature nicht implementiert hat]
Dokumentation	[z. B. Testergebnisse als Kommentar in GitIssue]

7.3. Nicht getestet

Es ist genauso wichtig zu dokumentieren, was bewusst NICHT getestet wird — und warum. Das verhindert Missverständnisse und zeigt, dass die Entscheidung bewusst getroffen wurde.

Bereich	Begründung
[z. B. Infrastruktur-Code (Docker, CI-Konfiguration)]	[z. B. Zu aufwändig für den Projektumfang; wird durch manuelle Überprüfung abgedeckt]
[z. B. Drittanbieter-Bibliotheken]	[z. B. Vertrauen in externe Qualitätssicherung der Bibliothek]
[Weiterer Bereich]	[Begründung]

7.4. Werkzeuge & Frameworks

Werkzeug	Zweck	Dokumentation
[z. B. JUnit 5]	Unit- und Integrationstests (Backend)	https://junit.org/junit5/
[z. B. Mockito]	Mocking von Abhängigkeiten in Unit-Tests	https://site.mockito.org/
[z. B. Testcontainers]	Datenbankintegrationstests mit echtem Container	https://testcontainers.com/
[z. B. Cypress]	End-to-End-Tests im Browser	https://www.cypress.io/
[z. B. JaCoCo]	Messung der Testabdeckung	https://www.jacoco.org/

7.5. Testabdeckung

Definiert die angestrebten Mindestwerte für die Testabdeckung.

Schicht	Zielwert	Bemerkung
Service-Schicht	[z. B. $\geq 80\%$]	Geschäftslogik hat höchste Priorität
Persistenz-Schicht	[z. B. $\geq 60\%$]	Grundlegende CRUD-Operationen abgedeckt
Controller / API	[z. B. $\geq 70\%$]	Wichtigste Endpunkte per Integrationstest abgedeckt
Frontend-Komponenten	[z. B. $\geq 50\%$]	Fokus auf wiederverwendbare Kernkomponenten



Abdeckung ist kein Selbstzweck. 80 % sinnlose Tests sind schlechter als 60 % gut durchdachte Tests. Qualität der Tests ist wichtiger als der Prozentwert.

7.6. Integration in den Entwicklungsprozess

7.6.1. Ausführung der Tests

Lokale Entwicklung

```
# Alle Unit-Tests ausführen
[z. B. pytest | npm test]

# Testabdeckung prüfen
[z. B. pytest --cov]
```

CI-Pipeline

Tests werden automatisch bei jedem Push im Repository ausgeführt. Je nach Testart können unterschiedliche Trigger und Umgebungen sinnvoll sein. Die Konfiguration der CI-Pipeline sollte sicherstellen, dass alle Tests zuverlässig und schnell ausgeführt werden.

7.6.2. Definition of Done

Folgende Testkriterien sind Teil unserer Definition of Done:

- ☐ Alle bestehenden Tests sind grün
- ☐ Neue Funktionalität ist durch Unit-Tests abgedeckt
- ☐ Die Mindest-Testabdeckung wird nicht unterschritten
- ☐ Manuelle Tests für neue Features wurden durchgeführt
- ☐ Keine bekannten kritischen Fehler sind offen

7.7. Fehlerbehandlung & Bugtracking

Aspekt	Beschreibung
Wo werden Bugs erfasst?	[z. B. Als Issue mit Label "Bug" in GitHub Issues]
Priorisierung	[z. B. kritische Bugs werden sofort im laufenden Sprint behoben; andere ins Backlog]
Reproduzierbarkeit	[z. B. Jeder Bug-Report enthält Schritte zur Reproduktion, erwartetes und tatsächliches Verhalten]

Dieses Testkonzept wird bei Bedarf aktualisiert — insbesondere wenn neue Testarten eingeführt, Werkzeuge gewechselt oder Abdeckungsziele angepasst werden.

8. Deployment & Betrieb: F2 - EnergyBalance

=

Dieses Dokument beschreibt, wie das System gebaut, ausgeliefert und betrieben wird: lokal, in der CI/CD-Pipeline und in den verschiedenen Umgebungen. Es richtet sich primär an Entwickler:innen und deckt das ab, was nach dem Schreiben von Code passiert.

Technische Entscheidungen rund um Containerisierung und CI/CD (das **Warum**) sind als ADRs im [Architekturdokument](#) (`architecture.adoc`) dokumentiert. Dieses Dokument beschreibt das **Wie** und **Was**.

8.1. Umgebungsübersicht

Das System wird in mehreren Umgebungen betrieben, die unterschiedliche Zwecke haben. Jede Umgebung ist vollständig isoliert. Eine Änderung in einer Umgebung beeinflusst keine andere.

Umgebung	URL	Zweck	Branch	Deployment
Lokal	http://localhost:5173 (Frontend) http://localhost:3000 (Backend)	Entwicklung und manuelle Tests auf dem eigenen Rechner. React/Vite Frontend, NestJS Backend und Supabase-Anbindung werden lokal getestet.	Beliebig	Manuell über 1. Terminal npm run dev (frontend), 2. Terminal npm star :dev (backend)
CI	—	Automatische Qualitätsprüfung bei jedem Push	Alle Branches	Automatisch (GitHub Actions)
Staging	—	Integrationstest; Abnahme durch Product Owner vor Release	main oder develop	Automatisch nach Merge in main
Produktion	—	Live-System für Endnutzer:innen	main (getaggt)	Manuell mit Freigabe



In der Hochschulumgebung entfällt oft eine separate Produktionsumgebung. In diesem Fall dient Staging gleichzeitig als Produktionsumgebung.

8.2. Lokale Entwicklungsumgebung

Diese Anleitung richtet sich an neue Teammitglieder, die das System zum ersten Mal lokal zum Laufen bringen wollen.

8.2.1. Voraussetzungen

Werkzeug	Version	Installationshinweis
Node.js	≥ v22.12.0 (LTS)	https://nodejs.org/en/download — benötigt für React/Vite und npm
React Native + Vite	≥ v8 0.16	Download über Node.js
npm	≥ 10.0	Wird automatisch mit Node.js installiert
Git	Aktuell	https://git-scm.com — Versionsverwaltung
VS Code	Aktuell	Entwicklungsumgebung für Frontend/Backend
Supabase Account	Aktuell	https://supabase.com — PostgreSQL Datenbank + Auth

8.2.2. System lokal starten

```
# 1. Repository klonen
git clone [Repository-URL]
cd PWA-SE

# 2. Frontend-Abhängigkeiten installieren
cd frontend
npm install
npm install react-icons

# 3. Frontend starten
npm run dev

# 4. Backend-Abhängigkeiten installieren
cd backend
npm install @nestjs/mapped-types

# 5. Backend starten
npm start :dev

# Frontend erreichbar:
# http://localhost:5173
```

8.2.3. System stoppen und aufräumen

```
#Anwendung stoppen
STRG + C
```

```
# Abhängigkeiten neu installieren (falls Fehler auftreten)
Remove-Item -Recurse -Force node_modules
Remove-Item -Force package-lock.json

npm install
npm run dev
```

8.2.4. Häufige Probleme beim lokalen Start

Problem	Lösung
Port bereits belegt (address already in use)	Anwendung stoppen oder belegten Port prüfen: netstat -ano (Windows)
Datenbank-Container startet nicht	Supabase-Verbindung prüfen und Konfiguration kontrollieren
Umgebungsvariablen fehlen	Prüfen ob <code>.env</code> -Datei vorhanden ist; Beispielwerte in <code>.env.example</code> ansehen
Node-Version passt nicht	Node-Version mit <code>node -v</code> prüfen und ggf. aktuelle Version installieren

8.3. Docker & Container

Dieser Abschnitt beschreibt, wie das System containerisiert ist, welche Container existieren, wie sie konfiguriert werden, und wie sie zusammenarbeiten.

8.3.1. Container-Übersicht

Service	Image	Zweck	Port (intern → extern)
frontend	—	—	5173 → 5173
backend	—	—	3000 → 3000
supabase	—	PostgreSQL-Datenbank + Authentifizierung über Supabase Cloud	extern verwaltet

8.3.2. Umgebungsvariablen

Alle Konfiguration erfolgt über Umgebungsvariablen und **nicht** über feste Werte im Code. Eine vollständige Liste mit Beispielwerten befindet sich in `.env.example`.

Variable	Beispielwert	Beschreibung	Pflicht
DATABASE_URL	jdbc:postgresql://[supabase-host]:5432/postgres	Verbindungs-URL zur Supabase PostgreSQL-Datenbank	Ja
DATABASE_USER	Project name	Datenbankbenutzer von Supabase	Ja
DATABASE_PASSWORD	[sicheres Passwort]	Datenbankpasswort — nie im Repository speichern	Ja
JWT_SECRET	[min. 32 Zeichen]	Geheimnis für JWT-Token-Signierung, falls eigene JWTs genutzt werden	Ja
SMTP_HOST	mailhog (lokal) / smtp.example.com (prod)	E-Mail-Serveradresse, falls E-Mails später genutzt werden	Ja
LOG_LEVEL	INFO	Log-Level: DEBUG / INFO / WARN / ERROR	Nein (Standard: INFO)
SUPABASE_URL	https://fwshawn-hufoldhkyycor.supabase.co	URL des Supabase-Projekts	Ja
FRONTEND_URL	http://localhost:5173	URL des Frontends für lokale Entwicklung	Ja
BACKEND_URL	http://localhost:3000	URL des Backends für lokale Entwicklung	Ja



Produktions-Secrets (Passwörter, API-Keys, JWT-Secrets) werden **nie** im Repository gespeichert — auch nicht in `.env`-Dateien. Sie werden ausschließlich als CI/CD-Secrets oder über einen Secret-Manager verwaltet.

8.4. CI/CD-Pipeline

Die Pipeline automatisiert Qualitätsprüfung und Deployment. Sie stellt sicher, dass kein Code in `main` gelangt, der Tests bricht oder Qualitätsstandards verletzt.

8.4.1. Pipeline-Schritte im Detail

Schritt	Beschreibung	Schlägt fehl bei	Ausgeführt bei
Linting	Code-Stil und statische Analyse prüfen	Stilfehler oder Warnungen über Schwellwert	Jedem Push
Unit-Tests	Alle Unit-Tests ausführen	Fehlschlagende Tests	Jedem Push

Schritt	Beschreibung	Schlägt fehl bei	Ausgeführt bei
Integrationstests	Tests gegen echte Datenbank (Testcontainers)	Fehlschlagende Tests	Jedem Push
Abdeckungsprüfung	Testabdeckung messen und mit Mindestwert vergleichen	Unterschreitung des Mindestwerts (siehe Testkonzept)	Jedem Push
Image-Build	Frontend- und Backend-Build ausführen	Build-Fehler	Jedem Push auf main und PRs
Staging-Deployment	Neue Images auf Staging-Umgebung ausrollen	Deployment-Fehler oder fehlschlagende Smoke Tests	Merge in main
Produktions-Deployment	Neues Release auf Produktion ausrollen	Deployment-Fehler oder fehlschlagende Smoke Tests	Manuell nach Tag-Erstellung

8.4.2. Pipeline-Konfiguration

Die vollständige Pipeline-Konfiguration befindet sich in:

```
.github/workflows/
├── ci.yml      ← Qualitätsprüfung (bei jedem Push)
└── deploy.yml ← Deployment (Staging & Produktion)
```

8.5. Deployment-Ablauf

8.5.1. Deployment auf Staging (automatisch)

Jeder Merge in **main** löst automatisch ein Deployment auf Staging aus. Kein manueller Eingriff erforderlich.

1. GitHub Actions baut das Frontend/Backend
2. Tests werden ausgeführt
3. Frontend wird z. B. auf Vercel deployed
4. Backend wird z. B. auf Render/Railway deployed
5. Team wird bei Erfolg oder Fehler benachrichtigt

8.5.2. Release und Deployment auf Produktion (manuell)

```
# 1. Sicherstellen, dass main aktuell und stabil ist
git checkout main
git pull
```

```
# 2. Release-Tag erstellen (Semantic Versioning)
git tag v1.2.0
git push --tags

# 3. In GitHub Actions: Produktions-Deployment manuell freigeben
# → GitHub → Actions → deploy.yml → "Run workflow" → Version bestätigen
```



Wir verwenden *Semantic Versioning* mit dem Schema MAJOR.MINOR.PATCH: MAJOR bei inkompatiblen Änderungen, MINOR bei neuen Features (abwärtskompatibel), PATCH bei Bugfixes.

8.5.3. Smoke Tests nach Deployment

Nach jedem Deployment werden folgende Prüfungen ausgeführt:

Prüfung	Erwartetes Ergebnis
GET /actuator/health (Backend)	HTTP 200, Status UP
Startseite laden (Frontend)	HTTP 200, Ladezeit < 3 s
Login-Flow	Erfolgreicher Login mit Testnutzer möglich
Benutzerprofil laden	Profildaten werden erfolgreich aus Supabase geladen
Datenbankzugriff prüfen	Lesen und Schreiben von Daten in Supabase erfolgreich
Reflexion erstellen	Neuer Eintrag wird gespeichert und angezeigt

8.6. Rollback

Ein Rollback ist notwendig, wenn ein Deployment kritische Fehler in der Produktionsumgebung verursacht. Schnelles Handeln minimiert die Auswirkungen.

8.6.1. Wann wird ein Rollback durchgeführt?

- Kritische Fehler in Produktion nach einem Deployment
- Smoke Tests schlagen nach Deployment fehl
- Fehlerrate oder Antwortzeiten steigen signifikant an

8.6.2. Rollback-Ablauf

1. **Entscheidung treffen** — Tech Lead oder Product Owner entscheiden über Rollback
2. **Stakeholder informieren** — Team und relevante Stakeholder benachrichtigen
3. **Rollback ausführen** (eine von zwei Optionen):

```
# Option A: Letztes stabiles Tag erneut deployen
# → GitHub → Actions → deploy.yml → Version des letzten stabilen Tags eingeben

# Option B: Manuell das vorherige Image aktivieren
docker compose pull [service]:[vorherige-version]
docker compose up -d [service]
```

4. **Smoke Tests ausführen** — sicherstellen dass das System wieder stabil ist
5. **Ursache analysieren** — was hat das Problem verursacht?
6. **Nachbereitung dokumentieren** — im Ticket oder als Kommentar im ADR

8.7. Monitoring & Logging

Dieser Abschnitt beschreibt, wie der Zustand des Systems beobachtet und wie Fehler diagnostiziert werden können.

8.7.1. Health Checks

Endpunkt	Beschreibung	Erwartet
GET /actuator/health	Gesamtstatus des Systems inkl. Datenbankverbindung	HTTP 200, {"status": "UP"}
GET /actuator/info	Version und Build-Informationen	HTTP 200
GET /api/status	Prüft, ob Backend erreichbar ist	HTTP 200
GET /api/users/me	Prüft Login/Auth-Funktion	HTTP 200 bei gültigem Login

8.7.2. Logs

Aspekt	Beschreibung
Log-Format	Strukturiertes JSON-Logging; enthält Zeitstempel, Level, Service, Message
Log-Zugriff (lokal)	<code>docker compose logs -f [service]</code>
Log-Zugriff (Staging/Prod)	[z. B. <code>ssh user@server → journalctl -u [service]</code> oder Logging-Dashboard-URL]
Was geloggt wird	Alle Fehler (ERROR), Warnungen (WARN), wichtige Geschäftsereignisse (INFO)
Was nicht geloggt wird	Passwörter, Tokens, personenbezogene Daten (DSGVO-Anforderung)

Dieses Dokument ist ein lebendes Dokument. Änderungen an der Pipeline, den Umgebungen oder dem Deployment-Prozess sind hier im selben Pull Request zu dokumentieren, in dem die technische Änderung vorgenommen wird.